IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Manuscript submitted for review; publication dates are assigned by the journal.

Digital Object Identifier to be assigned by the journal

STRATA: A Fixed-Buffer Scoped Symbol Table with LIFO Undo and Bounded Probing

ADHYAN JAIN[1], SHUBHI SINGH[1], and BHUVANESWARI M.[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

Corresponding author: Bhuvaneswari M. (e-mail: m.bhuvaneswari@vit.ac.in). Other e-mail addresses: adhyan.jain2024@vitstudent.ac.in (A. Jain), shubhi.singh2024@vitstudent.ac.in (S. Singh).

This work received no external funding. The authors declare no competing interests.

ABSTRACT A compiler that runs inside a fixed memory budget needs a symbol table whose size is fixed in advance, whose behavior at the limit is defined, and whose index work per call can be bounded on hostile source text. We present STRATA, a scoped symbol table that keeps an open-addressing index, packed identifier records and an undo stack in one caller-supplied buffer; a call that does not fit is refused and leaves the buffer unchanged. For every open-addressing rule that never moves a key, removing the newest key by clearing its cell restores the previous index exactly, and for linear probing last-in-first-out order is the only deletion order safe for every table. In an ideal random-hash model this gives the exact probability that a probe limit refuses the next fresh name; for a table of 65 536 slots at load three quarters a refusal probability of one in a million per request needs 258 probes, against 455 for a Chernoff-type tail bound. The law agrees with the implementation in 3.7 million trials at up to 16 384 index slots, and is consistent with the default hash on real identifiers, and we bound the refusal-filtered process, for insertion-only schedules, that the law does not describe exactly. On events logged by a C compiler for eight projects STRATA gave no wrong answer and needed a fixed buffer 16.4 to 17.9 percent smaller than a conventional single-buffer table that stores a hash word, but only 5.3 to 9.0 percent smaller than one that does not, a saving that comes from name packing. In repeated timing sessions on the final library it was slower than the table with a stored hash word by 5 to 18 percent on all eight traces, and slower than the one without by 4 to 13 percent on six of them. Bounded mode limits index slot examinations per declaration and lookup, not scope exit or the default-mode rebuild. Identifiers that collide for every seed slow the unkeyed table 300-fold at 32 768 names, growing quadratically; a keyed hash removes this effect for that collision family, and keeping honest names from being refused next to such names needs a fresh key per table, whose derivation is left to the caller.

INDEX TERMS Compilers, embedded software, hash tables, linear probing, memory management, symbol tables.

I. INTRODUCTION

A SCOPED symbol table maps program identifiers to their associated declarations and discards declarations belonging to an inner scope once that block terminates [1]. As one of the foundational data structures in compiler construction [2], its design on unconstrained workstations is widely considered an established problem: pairing an off-the-shelf hash map with an auxiliary undo stack typically suffices [3]. However, this conventional approach proves inadequate when the compiler or runtime system must execute within strictly constrained execution environments. Language front ends embedded on microcontrollers, inside secure boot loaders, or within static analysis tools operating under rigid memory quotas impose three fundamental requirements that traditional designs do not satisfy:

Bounded memory. The table must reside entirely within a predetermined memory region allocated prior to translation, avoiding dynamic memory allocators whose heap fragmentation and non-deterministic failure modes lie outside the compiler's control.

Defined behavior at the bound. When an insertion exceeds available storage, the table must report the failure cleanly while preserving total internal consistency. A partially mutated table risks turning an out-of-memory condition into silent, catastrophic compilation errors.

Bounded work on untrusted input. When processing third-party or untrusted source text, the operational cost of individual table accesses must not be vulnerable to pathological

identifier selections. Hash tables are well-known targets of algorithmic complexity attacks [4], and naive seed randomization on standard non-cryptographic hashes does not prevent systematic multicollisions [5].

The scoped symbol tables we inspected, the textbook undo-stack scheme, LLVM's `ScopedHashTable` and the Tiny C Compiler's symbol stacks [3], [6], [7], obtain memory from a general allocator, treat an allocation failure as fatal or leave it unspecified rather than returning a refusal with the table unchanged, and use unkeyed hashes. On the analytical side, while the theory of open addressing extensively covers insertions and deletions in arbitrary order [8]–[10], the structured last-in-first-out (LIFO) deletion sequence inherent to lexical scoping has not been analyzed as a distinct structural regime, and operational probe limits are typically chosen via tail bounds of uncharacterized looseness.

In this article, we investigate the concrete costs of simultaneously satisfying all three requirements and provide principled guidelines for configuring such tables. We present STRATA, a scoped symbol table encapsulated within a single caller-supplied buffer of 32-bit words. While its underlying algorithm employs the classical combination of a single hash index, packed records, and an undo stack for scope exit [3], our primary contributions lie in the unified single-buffer architecture, the formal proofs governing its structural and probabilistic invariants, and an extensive empirical evaluation:

1) **A single-buffer scoped symbol table with all-or-nothing refusal.** Hash index slots, interned identifier records with innermost bindings, hidden declarations, and scope markers share a single caller-supplied buffer. Any operation that exceeds available capacity returns an explicit status code and leaves the underlying buffer state byte-for-byte unchanged (Sections IV and VII).
2) **Exact undo and its converse.** We prove that for any non-reordering open-addressing scheme, removing the most recently inserted key simply by clearing its slot exactly reconstructs the preceding index state, ensuring that the index is always identical to an insertion-only table of live names. For any given table state, the safe deletion orders correspond precisely to the linear extensions of a dependency relation; furthermore, for linear probing, LIFO order is the only deletion order universally safe across all table states (Section V).
3) **An exact refusal law for bounded probing.** For a static index under probe limit L, we derive the exact finite-size probability that an honest identifier is refused under an ideal hashing model, validate this distribution empirically against the implementation, and establish formal bounds for the refusal-filtered process. We also demonstrate that a Chernoff-type bound on interval occupancy, the natural first choice, is loose by a factor growing as $L^{3/2}$ (Section VI).
4) **Empirical decomposition on compiler workloads.** Replaying trace events captured from C compilers across eight open-source benchmarks, we isolate the footprint gains of single-buffer operation from the compact layout itself. An exact space ledger shows that the saving comes from name packing and the omitted hash word, not from identifier interning; we measure the stronger baseline without a stored hash, and report that on the compiler traces STRATA is slower than the table with a stored hash word and, on most traces, than the one without (Section IX).
5) **Collision attacks, defenses, and key lifetimes.** We construct universal multicolliding C identifiers against the default hash mixer, quantify the resulting algorithmic slowdown alongside the overhead of keyed hashing, and establish the operational conditions under which fresh keyed instances defend against oblivious adversaries (Sections VI-C and IX-H).

We do not claim a new hashing scheme, a faster symbol table, or a result on hardware. STRATA is slower than a conventional single-buffer table; the refusal probability is classical as a quantity (the tail of the unsuccessful-search cost) and is credited as such, and what is new is its explicit finite-size sum, its validation against an implementation, the bounds for the refusal-filtered process and its use as a design rule; the collision construction applies a published technique [5]; and the microcontroller is emulated. Section XII explicitly details the boundary conditions and experimental limitations of our findings.

The remainder of this article is structured as follows. Section II reviews related work, and Section III formalizes the problem and threat model. Section IV presents the architecture of STRATA, Section V establishes the exact undo theorem, and Section VI derives the refusal law under bounded probing. Section VII details our verification methodology, Sections VIII and IX report the experimental setup and results, and Section X introduces the interactive visualization tool. Sections XI through XIII provide broader discussion, discuss limitations, and conclude.

II. RELATED WORK

A. SCOPED SYMBOL TABLES

Graham *et al.* [2] described hashed symbol tables for programming languages with explicit block structure. The architectural baseline adopted here follows Appel's classic formulation for block-structured languages [3]: a single central hash table maps each identifier to its innermost binding, an auxiliary stack tracks the bindings created within each scope, and scope exits pop the stack to restore shadowed declarations. This organization ensures that lookup latency remains independent of lexical nesting depth, in contrast to traditional cascades of per-scope maps [1]. In modern production software, LLVM's `ScopedHashTable` [6] offers a reusable implementation relying on heap-allocated nodes; lcc [11] links separate table structures per scope level, and the Tiny C Compiler [7] maintains symbol stacks popped upon scope exit. In the versions we inspected, all of them allocate per symbol or per node; TinyCC exits the process when an allocation fails and LLVM's allocators report a fatal error

by default, which is defined behavior but not a refusal after which the table can be used; and we found no documented bound on the work of a call. Region-based allocation [12] eliminates much of the general allocator overhead in compiler implementations; a chained table whose entries reside within a contiguous stack-like arena represents its natural single-buffer manifestation and serves as our primary baseline. In embedded execution environments, runtimes such as MicroPython intern identifiers into shared string pools [13], addressing identifier duplication but leaving lexical scoping and allocation failure handling unaddressed.

B. DELETION IN OPEN ADDRESSING

Evicting a key from an open-addressing table using linear probing typically necessitates sentinel tombstones or backward-shift displacements [8], [14], and the specific deletion strategy profoundly influences long-term cluster formation [9]. Sanders [15] established the exact conditions governing when an evicted entry requires a tombstone. Alternative schemes such as Robin Hood hashing [16] (employed in the predecessor to this work [17]), cuckoo hashing [18], and hopscotch hashing [19] actively relocate keys during insertion and deletion. While the fact that the most recently inserted key in linear probing can be removed simply by clearing its slot is a natural consequence of greedy probing (and occasionally leveraged in backtracking code), we formalize this property for all non-reordering schemes, characterize all safe deletion orders, and leverage this structural invariance for exact finite-size analysis. This behavior is conceptually related to, yet strictly weaker than, strong history independence [20], [21]: here, buffer layout is a deterministic function of the live identifiers and their insertion sequence.

C. ANALYSIS OF OPEN ADDRESSING

Under an ideal random hashing model, Knuth formulated the classical exact analysis of insertion-only linear probing [8]. The distribution of unsuccessful search costs—whose tail corresponds directly to the refusal probability analyzed in this work—has been characterized through exact generating functions and Poisson approximations [22]–[24], while the displacement distribution of Robin Hood hashing [25], [26] reappears in Section V as an orphan key distribution. Recent theoretical advances have characterized optimal probe complexities for non-reordering open addressing [10] and the performance of dynamic workloads [27]. Alharbi, Banderier, and Bouillaguet [28] analyzed bounded linear probing under a model where keys exceeding a probe threshold are discarded and redrawn; in contrast, our system models clean refusal without mutation under LIFO scope rollback, which is a different process. Pagh *et al.* [29] and Thorup [30] provided complementary analyses of linear probing under limited hash independence.

D. ALGORITHMIC COMPLEXITY ATTACKS

Crosby and Wallach [4] demonstrated that an attacker capable of generating hash collisions can degrade hash table performance from constant time to quadratic worst-case list traversals. Aumasson and Bernstein [5] showed that non-cryptographic hashes including MurmurHash2 and MurmurHash3 admit universal multicollisions that hold across all seed values, and proposed SipHash as a secure alternative; an accompanying presentation [31] detailed the specific construction for MurmurHash3. HalfSipHash configured with one compression and three finalization rounds is recognized in the Linux kernel as an efficient mitigation against hash-table flooding [32]. Adaptive adversaries against randomized data structures have long been studied [33], [34]. Production compilers handling trusted inputs commonly prioritize throughput over security: for example, the internal hash table of the Rust compiler is deliberately unkeyed [35]. Here, we explore the adversarial scenario, measuring both attack degradation and defensive countermeasures within a scoped symbol table.

E. SUMMARY OF DIFFERENCES

Table 1 summarizes the structural and operational properties of relevant symbol table architectures. Entries for external systems reflect findings from public documentation and source inspection of the referenced releases. Entries marked ‘‘no’’ indicate that no corresponding mechanism was identified. The refusal column separates systems that return a refusal and leave the table usable and unchanged from those that end the process on allocation failure (TinyCC, LLVM by default) or do not specify the state.

What is known, what is new, and why the combination matters. Every ingredient of STRATA is known: the hashed table with an undo stack [3], linear probing [8], region allocation [12], packed names, fingerprints, keyed hashing [5] and the distribution of the unsuccessful-search cost [8], [24]. We do not claim any of them. The contribution is that, to our knowledge and within the limited literature search stated in Section XII, they have not been put together so that each one makes another one usable, and that this yields guarantees none of them gives alone. Removing a scope’s names by clearing their cells (Theorem 1) is what allows the classical analysis of an *insertion-only* table to describe a table that is being edited all the time, and that is what turns a probe limit from a guess into a computable design value (Section VI-B); a probe limit without that analysis would be set by a loose bound, as it was in an earlier version of this study. The single buffer with reserve-before-write is what makes a refusal leave the table unchanged and makes that property testable byte for byte; a heap-backed table cannot offer a fixed bound, and the heap-backed tables we inspected do not return a refusal after which the table is known to be unchanged. The probe limit together with a keyed hash bounds the work of a call on hostile input, and the refusal law prices what that bound costs honest input. What we add beyond the combination is stated where it is proved or measured: the exact-undo theorem for every non-reordering rule and the characterization of the safe deletion orders, with the converse for linear probing (Section V); the explicit finite-size refusal sum, its valida-

tion against the implementation, and proved bounds for the process after refusals (Section VI); and an exact space ledger with a measured stronger baseline, which shows that most of the apparent saving is not due to the compact layout and that the compact layout costs time. The last point is a result against our own design, and we report it for that reason. Section XII lists what the evidence does not cover.

III. PROBLEM FORMULATION

A. OPERATIONS AND SEMANTICS

A scoped symbol table manages identifier bindings across a stack of lexical scopes, initialized with a single outermost global scope, supporting four core operations:

- `enter`: push a new innermost lexical scope.
- `exit`: terminate the current innermost scope, invalidating all declarations introduced within it; disallowed at the global scope.
- `declare`(x, v): bind identifier x to semantic value v in the innermost scope. If x is already declared at the current depth, its binding is overwritten; otherwise, a new declaration is introduced, shadowing any identically named declarations in outer scopes.
- `resolve`(x): retrieve the value associated with the innermost visible declaration of x, or report that no active binding exists.

An identifier is a non-empty byte sequence of at most 255 bytes. A semantic value comprises a 32-bit integer coupled with an 8-bit type tag. We denote by d the number of active declarations, $n \leq d$ the count of unique identifiers among them, s the number of open local scopes, and $w(x)$ the storage size of identifier x in 32-bit words.

B. REQUIREMENTS

R1 **Fixed storage.** All table structures must reside within a single caller-allocated buffer of C 32-bit words, executing entirely without calls to dynamic memory allocators.

R2 **All-or-nothing refusal.** Operations must execute atomically: any request that cannot be accommodated returns an explicit status code and leaves the complete internal state unmodified.

R3 **Correct scoping.** Every invocation of `resolve` returns the exact declaration dictated by lexical scoping semantics across all successfully committed operations.

R4 **Bounded index work per call.** In static-index (bounded) mode, an operational parameter L, independent of active declarations d or identifier names, strictly bounds the maximum number of index slots inspected during `declare` and `resolve`. This bound governs slot examinations rather than aggregate execution latency: `exit` inherently scales with the number of declarations in the closing scope, and default-mode table resizing scales with n. Section VI maps L to aggregate operation bounds.

R5 **Rare, quantified refusal of honest input.** Under R4, the probability that a benign declaration is refused when buffer capacity remains available must be mathematically characterized and negligible.

Requirements R1 through R3 are satisfied by STRATA in its fixed-buffer modes, with or without a static index, as well as by our arena baseline; the growable mode obtains and enlarges its buffer from an allocator, keeps R2 and R3, and does not satisfy R1. Requirement R4 is guaranteed by bounded mode; R5 is met under the ideal-hash model of Section VI, for an honest or keyed input, and not for an adversary against the unkeyed hash (Section IX-H).

C. THREAT MODEL

We assume an adversary who supplies the source text, exercising full control over identifier selection and operation ordering. The adversary possesses complete knowledge of the table implementation and hash function algorithms, but cannot predict a private 64-bit seed initialized by the host device from an unpredictable entropy source. We consider two threat models: an *oblivious* adversary, who fixes all identifiers prior to key generation, and an *adaptive* adversary, who observes runtime indicators (such as refusal occurrences or timing variations) to guide subsequent identifier choices within a single table session. The adversary aims to degrade lookup throughput into worst-case linear scans or induce spurious refusals of legitimate identifiers under bounded probing. Memory-safety vulnerabilities and hardware side channels are considered orthogonal; Section VI-C examines the consequences of direct probe-length disclosure. Finally, exhausting buffer memory through a sheer volume of distinct identifiers represents ordinary capacity consumption rather than an algorithmic flaw; requirement R2 ensures that the table remains robust and queryable following exhaustion.

IV. DESIGN OF STRATA

A. ARCHITECTURE AND LAYOUT

Fig. 1 illustrates the system architecture, and Fig. 2 depicts the internal buffer layout. All internal state resides within a contiguous array of 32-bit words. The *hash index* occupies the first m words (m being a power of two). An empty slot is represented by zero; an occupied slot stores an 8-bit hash fingerprint alongside a 24-bit relative word reference to an identifier record. Collisions are resolved via linear probing, maintaining a maximum load factor of $\alpha \leq 0.75$.

The *identifier records* are placed directly following the hash index, growing upward towards the end of the buffer with exactly one record allocated per distinct visible identifier. Each record stores the semantic value of the identifier's *innermost* visible binding, a 32-bit metadata word (encoding the record length in words, a raw-storage flag, a type tag, and lexical scope depth), followed by the encoded name words. Consequently, lookup terminates immediately upon matching the identifier record, with latency independent of how deeply shadowed the identifier may be. Identifiers adhering to standard C identifier syntax (alphanumeric characters and underscores) are compactly encoded at five characters per 32-bit word using 6-bit codes ranging from 1 to 63 (preserving zero

TABLE 1. Properties of Scoped Symbol Tables and Identifier Stores Relevant to This Work.

Design	Scopes with shadowing	One caller-supplied buffer	Defined refusal, state unchanged	No rebuild inside a call	Bound on probes, exact refusal law	Keyed hash
Undo-stack scheme [3], LLVM `ScopedHashTable` [6]	yes	no	no	no	no	no
Tiny C Compiler symbol stacks [7]	yes	no	no	no	no	no
MicroPython interned strings [13]	no	pools	not stated	n/a	no	no
METIS-X [17] (predecessor)	yes	no	no	no	no	no
Conventional arena table (our baseline)	yes	yes	yes	no	no	no
STRATA, fixed buffer, default	yes	yes	yes	no	no	optional
STRATA, growable mode	yes	no (allocator)	yes	no	no	optional
STRATA, fixed buffer, bounded	yes	yes	yes	yes	yes	optional

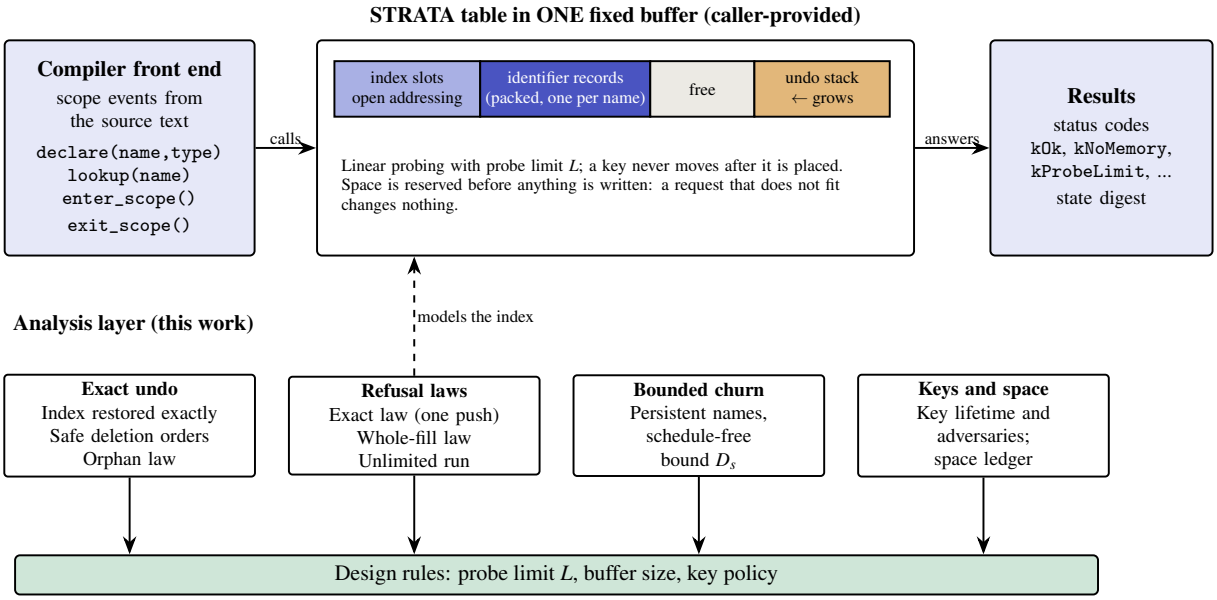


FIGURE 1. Architecture. The compiler front end issues four calls to a STRATA table that keeps its open-addressing index, packed identifier records and undo stack in one fixed buffer; the analysis of Sections V and VI turns the model of the index into design rules for the probe limit, the buffer size and the key policy.

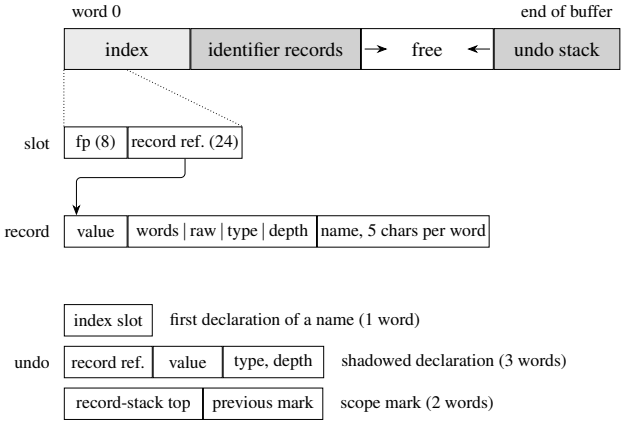


FIGURE 2. Layout of the buffer and of the three kinds of undo entries. The index and the identifier records grow from the start of the buffer, the undo stack from its end; the free space between them is shared.

TABLE 2. Bit-Level Encoding of the Buffer (32-Bit Words, Bit 0 Least Significant). References Count Words.

Item	Fields
Index slot (1 word)	bits 31–24 fingerprint (top byte of the hash, 1 if 0); 23–0 identifier reference; 0 = empty
Identifier record	word 0 value; word 1 meta; then $n \leq 64$ name words
Meta word	6–0 name words n; 7 raw flag; 9–8 raw length mod 4; 17–10 type tag; 31–18 scope depth ($\leq 16\,383$)
Undo, new name (1 word)	index slot, $< 2^{24}$, so bit 24 is clear
Undo, shadowing (3 words)	word 0 record reference with bit 24 set; word 1 saved value; word 2 saved type and depth (bits 31–10 of meta)
Scope mark (2 words)	word 0 identifier-stack top at entry; word 1 position of the previous mark

as a sentinel). A lookup packs the query once and compares it with stored names word by word. Arbitrary identifiers containing extended byte values are stored as raw bytes padded to 4-byte boundaries. Table 2 details the complete bit-level layout.

Packed identifiers avoid the need for separate length fields:

because character codes are strictly positive, zero padding in the terminal word cannot be misconstrued as valid characters, and identifiers of different lengths naturally diverge in word counts or terminal padding bits. Raw strings are copied byte by byte with zero padding, storing the length modulo 4 in the metadata word to preserve distinction between strings containing trailing nulls; the raw-format flag distinguishes raw from packed encodings of identical word length. Words are stored in native host byte order; consequently, serialized buffers are not binary-portable across different endianness platforms, and keyed hashing was validated on little-endian architectures. Identifier references are 24 bits wide with reference zero reserved ($itop$ begins at 1), capping the maximum addressable buffer size at $2^{24} - 1$ words (67 108 860 bytes, approximately 64 MiB). Because the identifier stack cannot exceed the 24-bit addressing boundary before exhausting physical buffer capacity, the explicit reference check in Algorithm 1 acts as a defensive safeguard that remained unreached across all empirical benchmarks. Callers are required to provide a 4-byte aligned memory region as part of the library contract.

The *undo stack* grows downward from the high end of the buffer. Declaring a previously unseen identifier pushes a single word recording the allocated index slot. Declaring an identifier that shadows an outer binding pushes three words: the record reference alongside the displaced value, type tag, and scope depth. Entering a scope pushes a two-word marker capturing the current top of the identifier stack and the location of the previous scope marker. For an index of m slots, total active buffer consumption is bounded by

$$M = m + 1 + \sum_{x} \big(2 + w(x)\big) + n + 3\,(d - n) + 2\,s \quad (1)$$

words, where the summation spans the n distinct visible identifiers and the additive constant accounts for reserved reference zero. In comparison, the conventional arena baseline (Section VIII-B) stores for each declaration a bucket link, full hash word, semantic value, combined length-depth word, and the unpacked identifier bytes.

B. DECLARATION AND LOOKUP

Algorithm 1 outlines the declaration pipeline. Following a strict *validate-search-reserve-write* lifecycle, the initial search phase is entirely read-only. The only operation capable of mutating buffer layout prior to the final refusal point is Ensure (Algorithm 3), which guarantees atomic completion without intermediate state corruption. An unsuccessful search naturally halts at the first unoccupied slot encountered along the identifier's probe sequence—identically where linear probing inserts fresh keys. This slot index is cached and reused, ensuring that successful declarations traverse the probe path exactly once. If Ensure triggers an index resize (which is disabled in bounded mode), the cached slot index is invalidated and a fresh insertion probe is conducted.

Status precedence. Error conditions are evaluated strictly in the order shown in Algorithm 1: table validity, empty string, excessive length, same-scope redeclaration (which requires no allocation), memory exhaustion (`no-memory`, covering reference overflow, load factor violation, or buffer deficit), and finally probe limit exhaustion (`probe-limit`). An identifier that is both overly long and exceeds available memory reports `name-too-long`; an operation that exceeds both memory capacity and the probe limit reports `no-memory`. Scope entry verifies validity, maximum nesting depth (`too-deep`), and memory; scope exit checks validity and non-zero depth (`no-scope`). Search strings are normalized into a stack-resident buffer (268 bytes: 64 name words and three header words) before performing mutations, ensuring safe operation even if the input string references data within the table's own buffer prior to relocation. Capacity reservations are evaluated using 64-bit integer arithmetic, with architectural boundaries (24-bit references, 255-byte length, 64-word names, and 16 383 scope levels) verified prior to writing.

Algorithm 1 Declare(x, v)

Input: name x (1 to 255 bytes), value and type v; current depth δ; probe limit L (bounded mode) or $L = \infty$
Output: a status; the state is unchanged unless the status is `ok` or `redeclared`

1: **if** table invalid **then**
2: **return** `no-memory`
3: **end if**
4: $K \leftarrow$ MakeKey(x); **if** $|x| = 0$ **return** `empty-name`; **if** $|x| > 255$ **return** `name-too-long`
5: $(r, e) \leftarrow$ Find(K, L) {read-only; record or first empty slot within L probes}
6: **if** $r \neq$ `nil` **and** $depth(r) = \delta$ **then**
7: overwrite value and type of r;
8: **return** `redeclared`
9: **else if** $r \neq$ `nil` **then**
10: **if not** Ensure$(3, 0)$ **then**
11: **return** `no-memory`
12: **end if**
13: push $(r \mid$ shadow bit, $value(r), type(r), depth(r))$; overwrite value, type, depth of r;
14: **return** `ok`
15: **end if**
16: **if** $itop + 2 + |K| > 2^{24} - 1$ **then**
17: **return** `no-memory`
18: **end if**
19: **if not** Ensure$(3 + |K|, 1)$ **then**
20: **return** `no-memory`
21: **end if** {may grow; all-or-nothing}
22: **if** $L < \infty$ **and** $e =$ `nil` **then**
23: **return** `probe-limit`
24: **end if** {last refusal point}
25: append the record for x with v and δ; store reference and fingerprint in e (or reinsert, if the index was rebuilt)
26: push the slot on the undo stack;
27: **return** `ok`

Identifier lookup packages and hashes the query name, probing sequentially from its home slot until finding a slot matching the fingerprint, metadata, and packed words, encountering an empty slot, or reaching the probe threshold L. Hash fingerprints function purely as early rejection filters; every candidate match is verified by comparing identifier words directly.

C. SCOPE ENTRY AND EXIT WITHOUT TOMBSTONES

Entering a scope reserves two words and pushes a scope marker, failing only upon nesting overflow or buffer exhaustion. Exiting a scope unwinds the undo stack down to the active scope marker in reverse chronological order (Algorithm 2, Fig. 3). Three-word shadowing entries reinstate the overwritten value, type, and depth fields. Single-word new-name entries directly clear their corresponding index slot. The identifier stack top is then restored from the marker, instantly reclaiming records allocated within that scope.

Simply clearing the slot is sound because of the strict LIFO removal order: being the most recently inserted key in the index, the evicted key cannot lie on the active probe path of any previously inserted live key. Section V proves this invariance for all non-reordering schemes and demonstrates that no alternative deletion order guarantees this property universally. Consequently, scopes close without writing tombstones or performing backward key shifts. Scope exit operations cannot fail due to memory exhaustion: they strictly deallocate storage and, in default growable mode, may optionally release heap blocks to the system allocator without compromising table validity if the request is declined.

D. ALL-OR-NOTHING REFUSAL

Every table operation validates arguments, verifies capacity reservations, and only mutates memory once success is guaranteed. In fixed-buffer mode, reservations reduce to straightforward pointer and counter arithmetic. When dynamic resizing is enabled, Ensure (Algorithm 3) secures the expanded memory block before initiating any data migration, ensuring that allocation failure aborts cleanly prior to state alteration. Once capacity is reserved, subsequent writes cannot fail. As a result, every refusal condition—insufficient buffer memory, nesting depth exceeding $16\,383$, empty or over-length names, or probe limit exhaustion—preserves the existing state completely unchanged (tested at every reservation boundary and exhaustively for sequences of nine operations in Section VII, not proved for all inputs).

E. GROWING THE INDEX

In default mode, the index doubles in size whenever inserting a key would violate the load factor threshold. Identifier records are migrated upward as a contiguous memory block, enabled by relative offset referencing, and the index is reconstructed by reinserting active identifiers in chronological order (oldest first), preserving the LIFO invariants required by Section IV-C. Because one-word undo entries store absolute index slot positions that shift during relocation, each identifier record and its corresponding undo entry temporarily swap word values prior to reindexing, restoring them during reinsertion. This eliminates secondary search passes and avoids auxiliary scratch allocations. In default mode, an operation triggering an index rebuild exhibits $O(n)$ latency; in bounded mode, static index sizing completely eliminates dynamic rebuilds.

Algorithm 2 Enter and Exit

```
1:  procedure Enter
2:  if table invalid then
3:      return no-memory
4:  end if
5:  if δ ≥ 16 383 then
6:      return too-deep
7:  end if
8:  if not Ensure(2, 0) then
9:      return no-memory
10: end if                                   {last refusal point}
11: push mark (itop, mark); mark ← its position; δ ← δ + 1;
12: return ok
13: procedure Exit
14: if table invalid then
15:     return no-memory
16: end if
17: if mark = 0 then
18:     return no-scope
19: end if                                   {last refusal point}
20: while undo stack above mark do
21:     ϵ ← top entry
22:     if ϵ is a shadow entry then
23:         restore value, type, depth of its record; pop 3 words
24:     else
25:         empty the index slot ϵ; pop 1 word
26:     end if
27: end while
28: (itop, mark) ← saved fields of the mark; pop 2 words;
    δ ← δ − 1;
29: return ok
```

V. EXACT UNDO AND SAFE DELETION ORDERS

Model the index as a ring of m cells. A rule is *non-reordering* if a key never moves after it is placed and the cell chosen for an insertion depends only on the set of occupied cells and on the key's own probe sequence; this covers linear, quadratic and double hashing and uniform probing, and excludes cuckoo, hopscotch and Robin Hood hashing. The operations are *push*, which inserts a name that is not live, and *pop*, which removes the most recently pushed live name by clearing its cell; a refused push changes nothing. In STRATA the pushes are the *first* declarations of names, since a shadowing declaration needs no index slot. For a sequence $a = (a_1, \ldots, a_t)$ of distinct names, $\mathrm{Ins}(a)$ is the table obtained by inserting them in this order into the empty table.

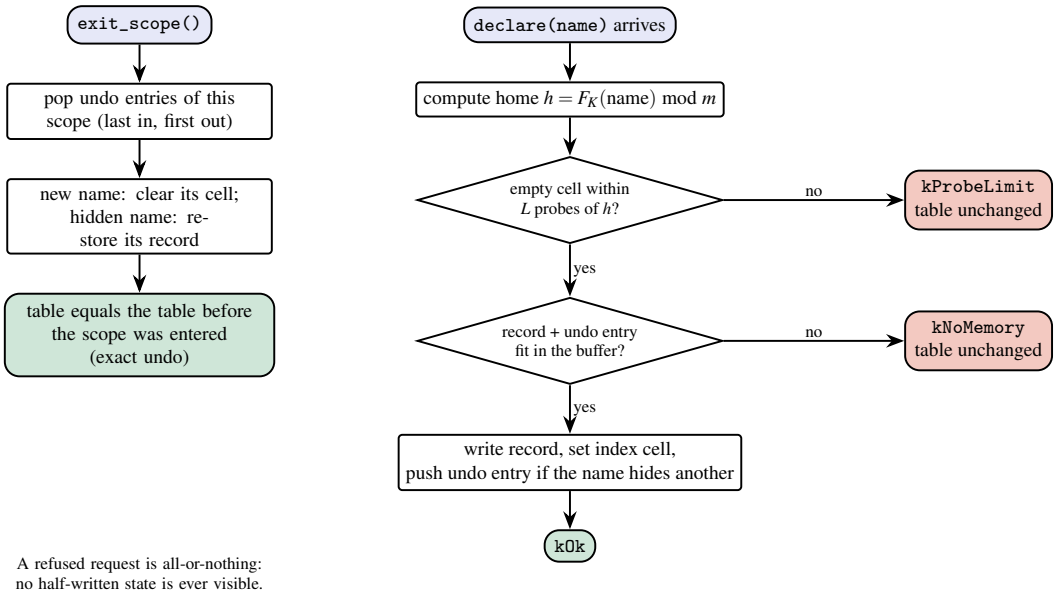


FIGURE 3. **Flow of a declaration and of a scope exit. A declaration is refused, with the table unchanged, when no empty slot is found within *L* probes or when the record and its undo entry do not fit; a scope exit pops the undo entries in reverse order.**

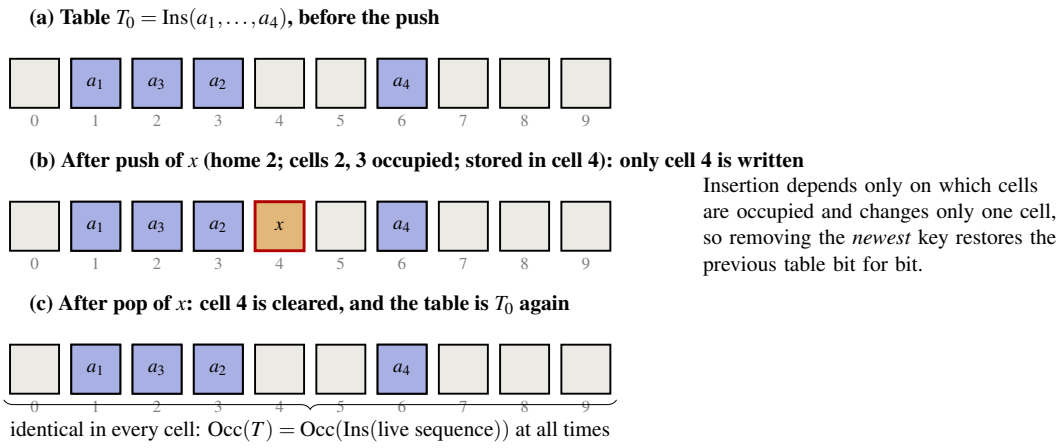


FIGURE 4. **Exact undo. Pushing *x* writes one empty cell; popping *x* clears it, and the table is identical in every cell to the table before the push.**

Theorem 1 (Exact undo): For every non-reordering rule, after any sequence of pushes and pops, with refused pushes leaving the state unchanged, the table equals $\text{Ins}(a)$, where a is the live sequence in push order.

Proof: Induction on the number of operations. A refused push changes nothing. A successful push of x onto $\text{Ins}(a)$ writes x into the cell that the rule chooses from the occupied set, which gives $\text{Ins}(a{\cdot}x)$ by definition. For a pop, $a = u{\cdot}x$ and the table is $\text{Ins}(u)$ with x additionally in the cell its insertion chose, because an insertion changes exactly one cell and that cell was empty in $\text{Ins}(u)$; clearing it gives $\text{Ins}(u)$ (Fig. 4). ■

The proof uses neither the lookup rule nor randomness, so it holds for adversarial names. What it proves is a statement about the *index cells*: the set of occupied cells and their contents equal those of $\text{Ins}(a)$. For STRATA the claim carries over to the bytes of the buffer only for the state that the operations define, and we state exactly which. We distinguish four levels, each tested over 66 582 scope exits in eleven configurations (fixed, bounded and growable buffers, odd sizes, tiny buffers that refuse). (a) *Logical state* (every accessor, the lookup of every name, the invariants): equal to a fresh table of the live declarations in all exits. (b) *Allocated region* (index, identifier stack, undo stack): equal to a fresh table with the same index size in all exits; the identifier stack alone is equal even when the index size differs, because references are relative, whereas the index words and one-word undo entries differ across index sizes by construction. In growable mode the index is not shrunk at once, so after a scope exit it can be larger than that of a fresh table built from the live names, and equality with such a table holds only when the

Algorithm 3 Ensure(w, a): room for w stack words and a new index entries

if static index (bounded mode) **then**
  return $(\text{names}+a)\cdot 4 \le \text{slots}\cdot\lambda$ **and** free words $\ge w$
end if {pure comparison}
$m' \leftarrow m$; **while** $(\text{names}+a)\cdot 4 > m'\lambda$ **do** $m' \leftarrow 2m'$ {$\lambda = 3$ (or 2, sparse)}
if $m' = m$ **and** free words $\ge w$ **then**
  return true
end if
$need \leftarrow m' + itop + undo + w$
if $need \le C$ **then**
  return Relocate(C, m')
end if {index grows inside the buffer}
if fixed buffer **then**
  return false
end if
$C' \leftarrow \min(1.5\,C \text{ or } need, \text{ limits})$; **if** $C' < need$ **return** false
return Relocate(C', m')
procedure Relocate(C', m') {the only mutating step}
obtain the new block (the single failure point; on failure return false); stash undo entries; move undo stack, then records; rebuild the index oldest first; update C, m, undo references; return true

index is configured to the same size. (c) *Every supplied word*: never equal, because removal is by moving counters: dead records, popped undo entries and scope marks remain in the free gap between the two stacks, and the high-water statistics are history. Only the removed name's index slot is cleared. (d) Against a fresh table with the same index and buffer size, the digest and the allocated region are equal in all exits and every differing word lies in the free gap. The proven claim is therefore (a) and (b) with equal configuration; a deployment that must not leave old identifiers readable in the free gap has to clear it itself. Two consequences matter below. The table does not age: no sequence of scope entries and exits lengthens probe paths. And in the random-function model, for a schedule that does not depend on the hash, the law of the index at every moment is that of an insertion-only table with the current number of live names, so classical insertion-only results [8], [25] and the refusal law of Section VI apply to the next push at every moment up to the first refusal; Section VI states what holds afterwards.

Why is last-in-first-out order special? For a table built by $\text{Ins}(x_1, \ldots, x_n)$ let $c(x)$ be the cell of x and say that y *depends on* x, written $y \lhd x$, if $c(x)$ lies strictly before $c(y)$ on the probe path of y (Fig. 5). A deletion order is *clear-safe* if, after every prefix of it has been cleared, every remaining key is still found.

Theorem 2 (Safe orders): Fix the homes of $x_1, \ldots, x_n$ $(2 \le n \le m)$ under linear probing. (a) A deletion order is clear-safe for these homes if and only if it is a linear extension of $\lhd$, that is, y is deleted before x whenever $y \lhd x$. (b) It is clear-safe for every assignment of homes if and only if it is the reverse insertion order.

TABLE 3. Limit Law of the Number of Keys That Depend on a Uniformly Random Live Key, $\mathbb{P}(\text{orph} = j)$, and Its Mean $\alpha/(2(1-\alpha))$.

α	j=0	1	2	3	4	mean
0.25	0.852	0.131	0.015	0.002	0.000	0.167
0.5	0.649	0.245	0.076	0.022	0.006	0.500
0.75	0.372	0.259	0.156	0.090	0.052	1.500
0.9	0.162	0.153	0.128	0.104	0.085	4.500

Proof: (a) Clearing never moves a key. The lookup of a remaining key y follows its probe path and stops at the first empty cell; the cells of that path before $c(y)$ were occupied, when y was inserted, by the keys x with $y \lhd x$. So y is found if and only if none of them has been cleared. (b) A key that depends on x is newer than x, so reverse insertion order is a linear extension of $\lhd$ for every assignment. Conversely, for $i < j$ give $x_1, \ldots, x_{j-1}$ distinct homes and give x_j the home of x_i; then $x_j \lhd x_i$, so a universally safe order deletes x_j before x_i for every pair $i < j$. ∎

A scheme is *greedy* if a key is stored in the first empty cell of its own probe sequence and a search for a key follows the same sequence and stops at the first empty cell; part (a) holds for every greedy scheme with these lookups, and the converse in part (b) is proved here for linear probing only. The number of keys that depend on a key quantifies the cost of leaving the discipline: it equals $Y_c - 1$, where Y_c follows the queue recursion $Y_c = (Y_{c-1} - 1)^+ + A_c$ with A_c the number of keys homed at cell c, and for $n/m \to \alpha$ its law converges to that of a discrete M/D/1 queue, with mean and fraction of independent keys

$$\mathbb{E}[\text{orph}] = \frac{\alpha}{2(1-\alpha)}, \qquad \mathbb{P}(\text{orph} = 0) \to \frac{(1-\alpha)(e^{\alpha}-1)}{\alpha}. \tag{2}$$

At load $3/4$ only 37 percent of keys could be removed out of order without harm, and clearing a random key orphans 1.5 keys on average (Table 3). The limit law is Janson's Robin Hood displacement law [25]; the reading as orphans of a deletion and the finite-size form are new as stated. This is why every deletion method for arbitrary order needs tombstones or shifting [8], [9], and why STRATA could not adopt a relocating deletion: its undo entries store index slots.

VI. BOUNDED MODE AND THE EXACT REFUSAL LAW

A. STATIC INDEX AND PROBE LIMIT

In the bounded mode the caller fixes the number of index slots m when the table is created. The index is never rebuilt; a declaration that would exceed the load limit is refused like one that exceeds the buffer. The caller may also set a probe limit L. A new name is inserted only if an empty slot lies among the first L slots of its probe path, and is refused with `probe-limit` otherwise (Algorithm 1). Since a stored name never moves and scope exit only empties slots, every stored name lies within L slots of its home, and a search may stop after L slots. One call thus examines at most L

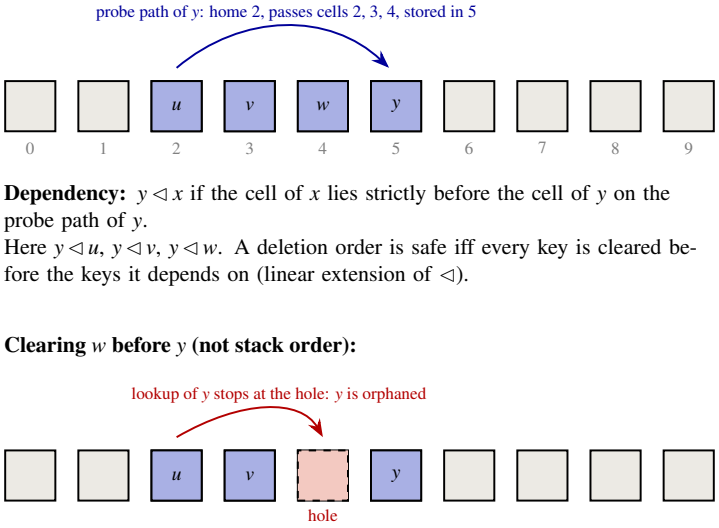


FIGURE 5. Dependency and orphans. Key *y* probed cells 2, 3 and 4 before it was stored in cell 5, so it depends on *u*, *v* and *w*. Clearing *w* first leaves a hole on its path and orphans *y*; clearing the newest key never does.

index words and, per fingerprint match, compares at most 64 name words (requirement R4). A name that hides a visible one needs no slot and is never refused for this reason.

From slot examinations to total work. A call reads its name once, at most 255 bytes, to pack and hash it (at most 51 packed or 64 raw words), examines at most L index words, and compares at most $\min(n_x, 64)$ name words for each of those whose fingerprint and meta word match, where n_x is the word count of the stored name. A declaration or lookup therefore performs at most $255 + L\,(1 + 64)$ elementary word operations whatever the names are; for $L = 258$ this is 17 025. The scratch space is one key of 268 bytes on the stack plus a handful of scalars, and no call is recursive. The bound excludes the scope exit, which is proportional to the number of declarations in the scope, the rebuild of the default mode, the generation of the key and the caller's own string handling, and, in growable mode, the allocator.

B. HOW OFTEN DOES THE LIMIT REFUSE AN HONEST NAME?

Probability model. Throughout this subsection the hash is an ideal random function: the homes of distinct names are independent and uniform on the m cells, and the schedule of operations does not depend on them. We consider an index of m cells holding $n = m - k$ live names, $k \geq 1$. By Theorem 1, the table without a probe limit is the insertion-only table of its live names at every moment, so the probability that the next fresh name needs more than L probes is that of an insertion-only linear-probing table with n keys. For $g \geq 1$ let $B(g) = g^{g-1}/g!$ and, for $j \geq 0$, $Q_j(s) = \frac{j}{s}\, s^{s-j}/(s-j)!$ for $s \geq j \geq 1$, with $Q_j(s) = 0$ for $s < j$ and $Q_0(s) = [s = 0]$. Four processes must be told apart, because the refusal rule makes the stored names depend on the hash.

(P1) *Ideal fill:* n fresh names are inserted into a table without a probe limit.

(P2) *First capped query:* the next fresh name is refused at limit L iff its probe count in (P1) exceeds L. This is what Theorem 3 computes, exactly.

(P3) *Conditioned fill:* the same query, given that none of the n earlier pushes would have been refused at limit L.

(P4) *Operational process:* a table with limit L that keeps receiving fresh names after refusing some of them; refused names are not stored.

The law is exact for (P2). It does not by itself describe (P3) or (P4), because the names that are accepted are selected by the refusal rule. Two facts give valid statements for them. For (P3), $\mathbb{P}(\text{refused} \wedge \text{clean fill}) \leq R_m(n, L)$, so the conditional probability is at most $R_m(n, L)/P_{\text{fill}}$, where $P_{\text{fill}} \geq 1 - \sum_{t<n} R_m(t, L)$ is the probability of a refusal-free fill (Appendix A). For (P4) with insertions only, couple the operational table B with the ideal table A that receives the same names: the occupied cells of B are always contained in those of A, since a name that B refuses leaves a cell empty that A fills, and a name that B accepts lands on or before the cell where A places it, a cell that is already occupied in A if it lies before. A refusal by B needs L occupied cells from the home, hence the same in A, so, since A holds every offered name, refused or not, the tth offered name sees at most $t - 1$ names

in A; if $N \le m$ fresh names are offered in all, the probability that the tth is refused is at most $R_m(t-1, L) \le R_m(N-1, L)$ whatever the earlier refusals, and the expected number of refusals is at most $N\,R_m(N-1, L)$. The bound is loose when many names are refused, because A then fills with names that B never stored. The coupling argument covers insertions only. With scope exits, a name accepted by B and popped may free a cell that A keeps occupied by a different name, the inclusion can fail, and for a schedule that oscillates after refusals we claim only the bound up to the first refusal; no experiment below covers scope exits after a refusal. The claims also require that the offered names are fixed independently of the hash (Section VI-C).

Theorem 3 (Exact refusal law): Let a ring of m cells hold $n = m - k$ keys, $k \ge 1$, with independent uniform homes under linear probing. The probability that a new key with an independent uniform home finds its first L cells occupied is

$$R_m(n, L) = \frac{k}{m} \sum_{g=L+1}^{n+1} (g-L)\,\pi_g, \qquad \pi_g = \frac{B(g)\,Q_{k-1}(m-g)}{Q_k(m)}. \tag{3}$$

Proof sketch: The sum ranges over block sizes $g \le n+1$. Cut the ring at its k empty cells. Each empty cell and the run of occupied cells after it form a block; by the cycle lemma [36] block sizes are Borel distributed, and, conditioned on filling the ring, the k blocks are exchangeable, so the first has size g with probability π_g by the Borel–Tanner identity [37]. A new key is refused exactly when its home starts a window of L occupied cells; a block of g cells contains $(g-L)^+$ such homes, and the expected number of blocks is k. The full proof is in Appendix A. ∎

$R_m(n, L)$ is nondecreasing in n (the occupied set of unrestricted linear probing does not depend on the insertion order and adding a key only adds a cell, so a window of L occupied cells persists), so the value at the load limit covers every lower load. Boundary cases are handled explicitly: for $k = 1$ the law reduces to $(m-L)^+/m$. It is evaluated in logarithms with the log-gamma function, because g^g and s^s overflow double precision at $m = 65\,536$, and it was compared with brute-force enumeration of every home sequence in exact rational arithmetic and with an independent derivation (Section VII). As $n/m \to \alpha$ it converges to $(1-\alpha)\sum_{g>L}(g-L)\,\mathrm{Borel}(g;\alpha)$, and with $\rho = \alpha e^{1-\alpha}$

$$R_\infty(\alpha, L) \sim \frac{(1-\alpha)\,\rho^{L+1}}{\alpha\sqrt{2\pi}\,(1-\rho)^2\,L^{3/2}} \qquad (L \to \infty). \tag{4}$$

The quantity is classical: it is the tail of the unsuccessful-search cost, whose generating function and Poisson limit are given by Janson and Viola [24] and whose finite-size values follow from Knuth's run-length counting [8]. What is new here is the short derivation of the explicit sum (3), its validation against the implementation, and its use as a design rule. A Chernoff bound on the number of homes in an interval, the natural first choice, gives $\rho^L/(1-\rho)$; by (4) it is loose by a factor growing like $L^{3/2}$. At load $3/4$ and $65\,536$ slots the bound exceeds the exact value by a factor of 315 at $L = 64$ and 4 637 at $L = 512$.

TABLE 4. Smallest Probe Limit L With Per-Insertion Refusal Probability at Most ε: Exact Law for Index Sizes m, Its Limit, and the Limit the Chernoff-Type Bound Would Require.

load	ε	m=2^8	2^{10}	2^{12}	2^{14}	2^{16}	$m \to \infty$	Chernoff
3/8	10^{-6}	25	28	29	29	29	29	43
3/8	10^{-9}	37	44	46	47	47	47	62
1/2	10^{-6}	42	49	51	52	52	52	81
1/2	10^{-9}	59	77	83	84	85	85	117
5/8	10^{-6}	70	93	101	104	104	104	171
5/8	10^{-9}	95	142	162	168	170	170	244
3/4	10^{-6}	121	201	242	255	258	260	455
3/4	10^{-9}	148	289	380	413	422	426	638

Table 4 turns the law into design values by bisection on L, which is valid because $R_m(n, L)$ is nonincreasing in L. At load $3/4$ and $\varepsilon = 10^{-6}$ the exact limit is 258 for $m = 2^{16}$ slots (the figure quoted in this article always refers to that index size; it is 260 as $m \to \infty$), against 455 for the Chernoff-type rule, 43 percent fewer probes allowed per call; it is smaller for small indexes (121 at 256 slots). The table counts $n = 3m/4$ live names; for $n = 3m/4 - 1$, the largest number for which a further name can still be admitted, a limit can be one probe smaller.

Per request and per compilation. The value ε is a probability per fresh-name request, not per compilation. A compilation that offers N fresh names has, for an oblivious schedule, a probability of at least one refusal of at most $N\,\varepsilon$ by the union bound (exactly, for a monotone fill, the theory supplement gives $1 - P_{\mathrm{fill}}$, which lowers the limit for a whole-fill target of 10^{-6} at $m = 4096$ from 402 to 306 probes), and for an insertion-only schedule the expected number of refused names is at most $N\,\varepsilon$ by the coupling above (no such bound is claimed for schedules with scope exits). For instance, $N = 10^4$ fresh names at $\varepsilon = 10^{-6}$ give at most 10^{-2} for the compilation; a target of 10^{-6} for the whole compilation needs $\varepsilon = 10^{-10}$. Repeated names do not consume slots, but a name that is popped and pushed again counts as a new request, and these requests are correlated; a retry of a refused name with an unchanged key and table is refused again, so retries add no independent chances. A schedule that adapts to observed refusals depends on the hash, and only the unconditional bound of L probes holds for it. The theory supplement also gives the probability that a persistent set of names can never refuse a fresh one and a schedule-free bound for bounded churn above persistent names [38].

C. KEYED HASH AND THE LIFETIME OF A KEY

The default hash is the 32-bit MurmurHash3 mixer [39] applied to the name words. As an option, the table hashes with HalfSipHash with one compression and three finalization rounds [5], [32] under a 64-bit key supplied by the caller, checked against the reference code [40]. Theorem 3 is a statement in the ideal model above. A keyed 32-bit hash with a 64-

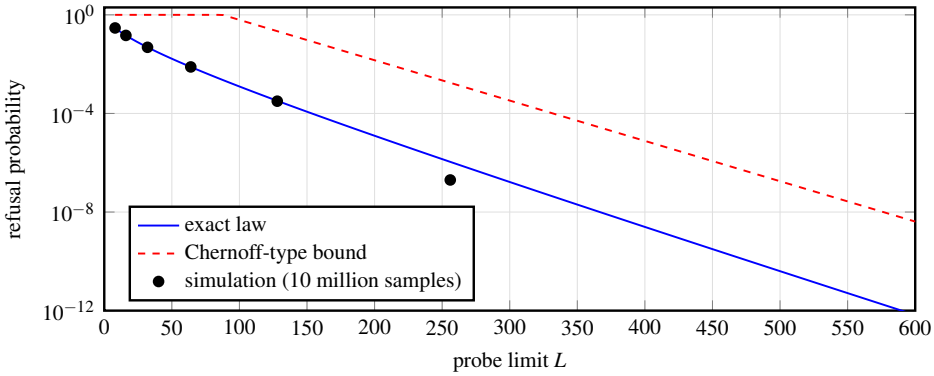


FIGURE 6. Probability that an insertion needs more than L probes ($m = 65\,536$, load $3/4$): exact law (Theorem 3), the Chernoff-type bound, and a stored simulation of 10 million samples from 500 tables.

bit key is not a random function, and fresh keys do not make the homes of all names exactly independent; using the law for the practical table relies on the assumption that HalfSipHash-1-3 under an unknown key is indistinguishable from a random function by the adversary, which is its design goal and not a theorem, and the empirical agreement of Section IX-I is evidence for names that are not adversarial. We make no cryptographic claim.

Under the assumption above, Theorem 3 applies to chosen names only if the key is independent of everything the adversary has seen. If every table is created with a fresh key and the adversary is oblivious, that is, fixes its names before the key is drawn, the homes are modeled as independent and uniform and the law is used for every name set and scope schedule. A fresh key per translation unit limits what can be learned across tables; it does not by itself address an adaptive adversary who queries the same table, whose refusals or probe lengths reveal information about the key. If one key is reused across tables and the adversary can observe probe lengths, it can learn names that collide with a target cell and plant a long run of occupied cells: in an experiment against the real table with an idealized probe-length oracle ($m = 2\,048$, $L = 512$, 700 honest names, five seeds), 1 to 5.5 percent of honest names on average (6.1 percent at most over seeds) were refused with a persistent key and none with a fresh key or with the same number of random names (Table 5). The rule is therefore one key per translation unit, derived from a device secret and a never-repeating counter, which must survive a restart of the device (a counter that restarts at zero repeats keys). The library only takes a caller-supplied 64-bit key; neither key derivation nor counter persistence is implemented, and our experiments use pseudo-random keys (flooding, refusal-law and adaptive runs) or one constant key (the keyed timing variant, the bounded traces and the embedded image), so the timing of the keyed variant says nothing about key derivation cost. The work bound of R4 does not depend on the key: a call examines at most L slots whatever the names. The experiment above uses an idealized probe-length oracle; it measures what an adversary could learn if such a channel existed, not a timing channel.

TABLE 5. Honest Names Refused by the Probe Limit (Percent of 700) After the Adversary's Learned Colliders Are Declared: Persistent Key (Mean and Maximum Over Five Seeds), Fresh Key, and Random Names.

colliders	persistent key (mean)	(max)	fresh key	random names
400	1.00%	1.29%	0.00%	0.00%
512	5.51%	6.14%	0.00%	0.00%
600	4.71%	6.00%	0.00%	0.00%
700	5.14%	5.71%	0.00%	0.00%
800	5.14%	6.00%	0.00%	0.00%

D. COLLIDING IDENTIFIERS FOR THE DEFAULT HASH

The mixer processes a name word k as $h \leftarrow \mathrm{rotl}_{13}\big(h \oplus f(k)\big) \cdot 5 + c$, where f multiplies, rotates and multiplies and is a bijection on 32-bit words that does not involve the seed. Following Aumasson and Bernstein [5], [31], flipping bit 18 of $f(k_1)$ flips bit 31 of the rotated word; multiplying by 5 and adding c leaves that difference in bit 31 for every h; and flipping bit 31 of $f(k_2)$ in the next word cancels it. Two consecutive words can thus be exchanged without changing the hash, whatever the seed. Here a word must be a packing of five identifier characters, and a random search finds word pairs that satisfy the constraints for both alternatives within a fraction of a second. Concatenating t such pairs gives 2^t distinct identifiers of $10t$ characters with one common hash; with $t = 15$ this yields 32 768 valid C identifiers of 150 characters, which collide under the built-in seed and eight further seeds.

VII. VERIFICATION

Unit and randomized differential tests. A test program compares the table with an independent reference model (a separate implementation of the semantics in terms of standard containers that shares no code with the table) after every operation, in configurations that cover packed and raw names, fixed and growable buffers, a dense and a sparse index, failing allocators, and a hook that forces every key into one probe cluster. It requires a 64-bit digest of the complete state to be

unchanged after every refusal and rebuilds a fresh table from the live declarations to compare memory. It makes 2 732 358 checks under AddressSanitizer and UndefinedBehaviorSanitizer with two compilers.

Further tests. (i) *Direct comparison on refusal:* 783 955 refusals were checked by comparing a saved byte-for-byte copy of the whole buffer, the options, the counters and the digest, not the digest alone; all were unchanged. The sweeps cover the capacity word by word, an exact threshold test (a call is accepted iff the free words reach the need: 529 probes at exactly the need accepted, 1 592 at one word less refused), growable budgets and failure of the allocator at each of its first twelve calls. (ii) *Status precedence* was printed and checked: an invalid table gives `no-memory`; `empty-name` and `name-too-long` precede any memory check; `too-deep` precedes `no-memory` on entry; `no-memory` precedes `probe-limit`; `no-scope` on exit at depth zero. (iii) *Encodings:* round trips of packed and raw names of every length from 1 to 255, names that differ only in padding or in embedded zero bytes, high-bit bytes, all 256 type tags and depth 16 383, with 1 030 constructed names pairwise distinct in both modes. (iv) *State equality* at the four levels of Section V. These tests found no defect in the library.

Bounded exhaustive enumeration. A second program starts every sequence from a freshly created table, enumerates every sequence of nine operations over {enter, exit, declare *a*, *b*, *c*, *d*} ($6^9 = 10\,077\,696$ per configuration) in eight small configurations (fixed buffers of 128 to 256 bytes or a static index of 16 slots, packed and raw names, a forced single probe cluster, the keyed hash, probe limits 0, 1 and 2) chosen so that every kind of refusal occurs, over the alphabet of the two scope operations and four declarations of different names (including a non-identifier name stored raw). The buffer is refilled with a marker pattern and the reference model is recreated for every sequence. After each operation the result and the lookups of all four names are compared with the model and the invariants are checked, and after each refusal the digest is compared (not the bytes). It counts each executed operation: 80 621 568 sequences, 725 594 112 operations, 130 361 632 of them refused (a refusal being a returned error status other than exit at the outermost scope), no failure. It was run again on the final code, after the changes described in Section IX-E, with identical counts (2 min 39 s). It is a finite check and not a proof of correctness.

Differential fuzzing and mutation. A libFuzzer harness ran 9.8 million executions on the code before the change of Section IX-F and 2.3 million on the final code, after the instruction-count changes of Section IX-E, without a failure. The mutation check detected each of 9 injected single-line defects, each within 2.4 seconds. After the last change to the library every test listed here was run again: the unit, differential and added tests with `g++` 16.2.1 and `clang` 22.1.8, with and without the sanitizers, and the enumeration of depth nine.

The analysis. Theorem 1 was checked in 488 919 state comparisons over five insertion rules, Theorem 2 on every assignment and every deletion order of small rings (397 562 pairs), and the refusal law against brute-force enumeration of every home sequence in exact rational arithmetic (258 cases) and against an independent derivation by Knuth's Abel-binomial route (481 cases). The proofs are pen-and-paper; no proof assistant was used.

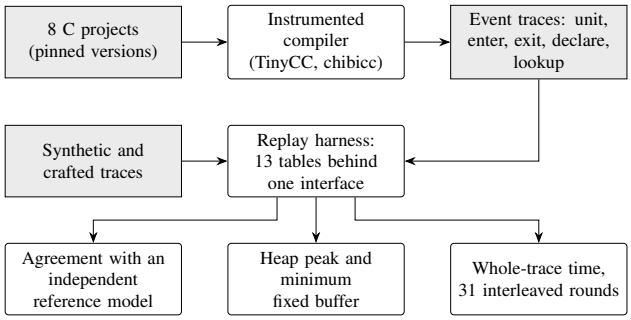


FIGURE 7. Evaluation method. Symbol-table events are logged by instrumented compilers and replayed on every table; an independent reference model decides correctness.

VIII. EXPERIMENTAL SETUP

A. WORKLOADS

The workloads are sequences of symbol-table events produced by a real compiler (Fig. 7). We added logging to the Tiny C Compiler [7] where it looks up an identifier, pushes a symbol, and opens or closes a scope, and compiled eight open-source C projects at pinned versions (Table 6). At the start of each translation unit every table is destroyed and created anew, as a compiler would do. *Trace semantics.* A trace is a text file with one event per line: a unit start, scope entry and exit, a declaration and a lookup with an identifier. The API has one namespace, and the traces log only the ordinary-identifier namespace of C (variables, functions, typedef names, enumeration constants). Struct, union and enum tags, members, labels and macros are separate tables of the compilers and are not logged. In TinyCC the hooks sit in `sym_find` (every lookup, including those that find nothing, which are 12.5 to 34 percent of the lookups), in `sym_link` for non-tag symbols, and in the block, prototype and function-parameter scopes; file-scope pushes made while a local scope is open (implicit declarations) are logged separately and ignored by every replay. A declaration of a name already declared in the same scope is replayed as the API's same-scope replacement; the eight TinyCC traces contain none, while the chibicc traces contain 1 to 122 per project. Empty names that occur in chibicc (9 324 lines) are dropped by the replay tools and stored as a single placeholder byte in the compiler integration. Sources are pinned by version tag or URL only; commit hashes and archive checksums were not recorded, and SHA-256 checksums of every trace file are published with the artifact. Three Mbed TLS units that TinyCC could not compile are skipped. The reference model that decides correctness is a separate implementation of the semantics above and shares no code with the table.

A second set of seven traces from an instrumented

TABLE 6. The Eight Compiler Traces. "Names" Counts Distinct Identifiers; "Peak" Is the Largest Number of Simultaneously Live Declarations.

Project	Units	Events	Declares	Lookups	Names	Peak	Depth
cJSON	2	20 874	5 247	9 194	1 366	1 067	10
littlefs	2	21 080	3 835	12 230	1 001	732	16
zlib	15	58 058	16 341	27 405	1 041	571	23
Lua	34	188 509	58 619	86 887	3 622	1 365	14
FreeRTOS	9	37 875	10 346	17 417	2 226	1 505	16
Mbed TLS	105	983 008	319 886	438 876	7 622	2 302	14
TinyCC	1	89 135	13 776	54 623	6 704	5 509	27
SQLite	1	268 960	27 072	172 426	7 801	4 245	22

chibicc [41] is used for the bounded-mode experiment, and nine synthetic traces stress cases that C programs rarely produce.

B. TABLES COMPARED

13 tables are driven through one interface.

- STRATA and four variants: raw instead of packed names, a sparse index (at most half full), no shrinking of the buffer, and the keyed hash.
- *The conventional arena table*, our main baseline: bucket heads, a stack of entries (chain link, hash, value, length and depth, unpacked name) and scope marks in one buffer, with the same storage discipline as STRATA. It was written as a strong conventional design, so that a comparison isolates the compact layout from the mere absence of an allocator; Section XII states how far it is from independent.
- One standard-library hash map per scope; an undo log over a standard-library map (the algorithm of LLVM's `ScopedHashTable`); the same over a Swiss-table map [42]; an interning Swiss-table map.
- METIS-X [17], the predecessor, a Robin Hood table with one slot per declaration, and two further tables from its repository, used unmodified.

C. METRICS AND PROCEDURE

Correctness is agreement with the reference model on every lookup and declaration result. *Heap peak* counts every allocation with its usable size. The *smallest fixed buffer* is found by bisection, at a resolution of 64 bytes, as the smallest buffer with which the whole trace replays without a refusal. *Replay time* is the time to replay a whole trace on a fresh table. In each of 31 measured rounds, after two warm-up rounds, every table replays the trace once and the starting table rotates. For two tables we report the median over rounds of the ratio of their times (pairing is by round: both tables are replayed in the same round) with a 95 percent bootstrap interval from 2000 resamples of rounds of the percent change $100\,(r-1)$, and call a difference *established* when the interval excludes zero ("n.e." otherwise); an established difference is not an equivalence claim, and a non-established one is not evidence of equal speed. Section IX-E repeats the comparison in independent sessions with an equivalence test.

D. ENVIRONMENT

All host measurements were taken on one machine: Intel Core Ultra 7 258V, Windows Subsystem for Linux 2 with Ubuntu 22.04 (Linux 6.18.33.2-microsoft-standard-WSL2), g++ 11.4.0 with `-O2`, the benchmark pinned to one core by `taskset` (the chibicc end-to-end driver is not pinned), run `20261013-143828-paper` of the public artifact [38]. The repeated sessions of Section IX-E ran on a different, shared native Linux machine described there. The embedded experiment uses `arm-none-eabi-g++` 10.3.1 with `-Os` and QEMU [43] 6.2 emulating a Cortex-M3 board (`mps2-an385`) restricted to 64 KiB of memory, without heap and without operating system. The refusal law was checked against the real bounded-mode table, with the keyed hash and a fresh random key per trial, in ten settings of 400 000 trials (100 000 at the largest index).

Which library each measurement used. The library changed during this study, and the measurements were not all repeated. The main benchmark run (memory, replay time, latency, ablation, synthetic traces, the sweep, the unit fit and the chibicc experiment), the hardening runs without honest names, the bounded-mode traces and the emulated microcontroller were made with earlier code. The smallest-buffer measurement was repeated on the final library and is identical for every table and trace, because the changes did not alter the layout. The flood was repeated: the numbers of refused names and the longest probe sequences are identical, and the time ratio of colliding to ordinary names was 365 instead of 300 at 32 768 names, which is within the run-to-run variation of timings. The five timing sessions, the instruction counts, the flood with honest names, the refusal-law checks and all tests used the final library. The library changes lowered the instruction count of STRATA by 6 to 8 percent, so the library version is a further difference between the first run and the sessions.

IX. RESULTS

A. CORRECTNESS

All 13 tables agree with the reference model on the eight Tiny C Compiler traces and on the seven chibicc traces: 195 table and trace pairs, no wrong lookup, no wrong declaration result, no refused declaration. On the synthetic traces METIS-X refuses 149 of 400 declarations when one name is shadowed in 400 nested scopes.

B. MEMORY

STRATA replays the traces in fixed buffers of 15.8 to 156.5 KiB, which is 27 to 36 bytes per live declaration at the peak (Fig. 8, Table 7). The conventional arena table needs a buffer that is larger on every trace, so that STRATA saves 16.4 to 17.9 percent; Section IX-C explains where this saving comes from.

Against tables that allocate, the difference is larger and has a different cause. The heap peak of STRATA in growable mode is 23 to 76 percent (median 58 percent) below that of METIS-X, and STRATA makes 98.7 to 99.9 percent fewer

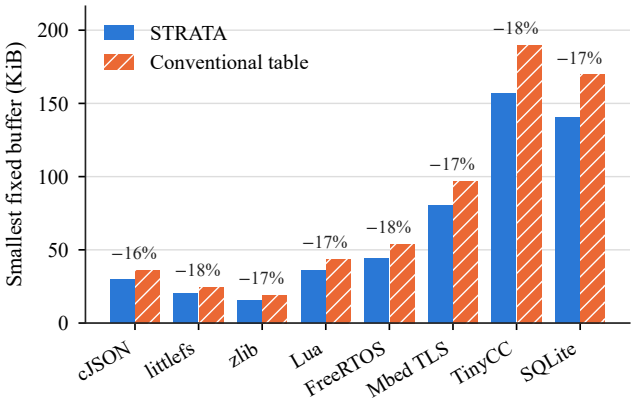


FIGURE 8. Smallest fixed buffer with which each compiler trace replays without a refusal. The label gives the saving of STRATA over the conventional arena table.

allocator calls. In growable mode the conventional arena table reaches the same heap peak as STRATA on 6 of the 8 traces, because both grow one block in steps. Most of the gain over the predecessor is therefore due to keeping everything in one allocation-free buffer, not to the compact layout.

C. ANATOMY OF THE SAVING: AN EXACT SPACE LEDGER

Every single-buffer layout considered here uses a number of words that is a sum of per-element costs, as in (1). Let C_0 be the conventional layout, C_1 the same without the stored hash word, C_2 an open-addressed index over one entry per declaration (a layout rule evaluated by accounting only, no code), S_{raw} STRATA's layout with raw names and S STRATA itself. C_1 is also implemented and measured (below). Subtracting the formulas gives exact identities at every instant:

$$U_{C_0} - U_{C_1} = d, \qquad U_{S_{\mathrm{raw}}} - U_S = \sum_{x \text{ packable}} \left(\left\lceil \frac{\ell_x}{4} \right\rceil - \left\lceil \frac{\ell_x}{5} \right\rceil \right), \tag{5}$$

and interning with an undo log saves the names of hidden declarations but costs one extra word per open scope. The accounting reproduces all 24 measured minimum buffers to the 64-byte resolution of the measurement. On the eight traces, dropping the stored hash saves 9.2 to 12.2 percent and packing a further 5.3 to 9.0 percent, while interning changes the footprint by between −0.38 and 0.00 percent (Fig. 9, Table 9).

The stronger baseline, measured. We implemented C_1 as a real table: the conventional arena table with the same hash, growth step and refusal behavior, entries of $3 + w$ words, and the hash recomputed from the stored name wherever the stored word was used. Its measured smallest buffer equals the ledger's prediction exactly on all eight traces, and at that buffer all four tables agree with the reference model with no refusal, while 64 bytes less makes each refuse. Against C_1, STRATA needs 5.3 to 9.0 percent less (7.5 percent pooled), against C_0 16.4 to 17.9 percent (17.3 percent pooled; Table 8); STRATA with raw names needs the same buffer as C_1 to within 64 bytes, so the remaining saving is name packing. The headline saving of one sixth is therefore a comparison with a conventional table that stores a hash word, and the saving against the stronger baseline is about one thirteenth. The smaller baseline costs time: C_1 replays 3 to 9 percent slower than C_0 (Section IX-E). Interning pays only in programs that shadow often, as the sweep of Section IX-J confirms.

What is counted. The buffer is the single caller-supplied block: for STRATA the index, the identifier records and the undo and scope stacks; for the arena tables the bucket heads, the entry stack and the scope marks. Not counted are the control structure (144 bytes for STRATA, 48 for each arena table), the stack key of one operation (268 and 256 bytes), a 256-byte static code table in STRATA, and the compiler's own state; in fixed mode no heap is used. Including them adds a constant of about 100 bytes in favor of the arena tables. The saving is that of the symbol table alone, not of a compiler. The bisection assumes that success is monotone in the capacity; we tested this on all eight traces and the four fixed-buffer tables (about 120 000 capacities: every 64 bytes up to twice the minimum, every 4 bytes within ±2 KiB of it, every 4 KiB up to eight times): no non-monotone point and no wrong result, and the threshold is sharp, at exactly the peak number of words in use. This holds for fixed mode on these traces; the STRATA index size is fixed by the buffer there. The measurement resolution of 64 bytes is under 0.5 percent of the smallest buffer. The peak is taken over live declarations, so the denominator of bytes per declaration is the peak of simultaneously live declarations, not of distinct names.

D. REPLAY TIME

Table 10 gives the paired comparisons of the first run. Against the conventional arena table, STRATA is faster on 0 traces, slower on 2, and on 6 the difference is not established; the point estimates lie between −10.6 and +10.7 percent. The compact table is not faster than the conventional one, and where a difference is established it is at most 10.7 percent. Against METIS-X, STRATA is faster on 8 of 8 traces, by 15.9 to 36.7 percent, and the conventional arena table is faster than METIS-X on 8 of 8, by 14.6 to 34.8 percent. Per operation (Table 11), the 95th-percentile latency of a declaration is 56 to 66 percent lower with STRATA than with METIS-X and that of a scope exit 71 to 82 percent lower; lookup latency differs by between −20.3 and +4.9 percent without a consistent direction.

E. REPEATED SESSIONS ON A SECOND CONFIGURATION

Because the verdicts of single runs changed between runs, we repeated the replay-time comparison in five independent sessions: a new process per trace, 31 paired rounds after two warm-up rounds as before, the start order rotated by session, the process pinned to one core, sessions five minutes apart, with the stronger baseline C_1 added. The sessions ran on a different configuration from the first run: a native Linux desktop (kernel 7.2.4, `g++` 16.2.1, Intel Core i7-13620H, `intel_pstate` governor `powersave`, turbo enabled), not WSL2 with `g++` 11.4. The machine is shared and not

TABLE 7. Memory on the Eight Compiler Traces. Smallest Fixed Buffer of the Two Single-Buffer Tables, and Heap Peak in Growable Mode of Six Tables (KiB).

	Smallest fixed buffer				Heap peak, growable mode (KiB)					
Project	STRATA (KiB)	Arena (KiB)	Saving (%)	Bytes per live decl.	STRATA	Arena	METIS-X	Map per scope	Undo log	Interning map
cJSON	30.0	35.9	16.4	28.8	42.9	42.8	102.7	72.0	116.0	140.4
littlefs	20.0	24.2	17.5	28.0	25.8	25.7	106.9	54.1	67.8	141.9
zlib	15.8	18.9	16.6	28.2	17.2	25.7	52.6	44.4	60.3	71.1
Lua	35.9	43.4	17.3	27.0	64.2	96.2	106.8	96.7	127.7	142.3
FreeRTOS	44.0	53.6	17.9	29.9	57.8	57.7	217.3	116.1	147.3	293.1
Mbed TLS	80.0	96.9	17.4	35.6	144.3	144.2	234.4	189.4	277.7	310.9
TinyCC	156.5	189.8	17.5	29.1	324.5	324.4	423.0	424.5	550.8	575.6
SQLite	140.4	169.4	17.2	33.9	194.8	194.7	451.0	333.3	513.5	1125.6

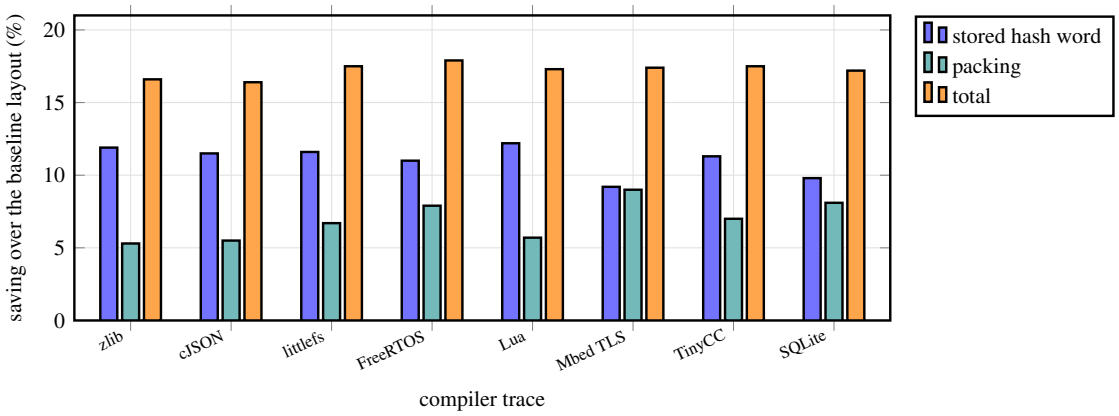


FIGURE 9. Saving of STRATA's layout over the conventional single-buffer layout on the eight traces, split into the stored hash word and name packing, from the exact space ledger.

TABLE 8. Smallest Fixed Buffer (KiB, 64-Byte Resolution) of STRATA, the Conventional Arena Table With (C_0) and Without (C_1) the Stored Hash Word and STRATA With Raw Names, and the Saving of STRATA Over C_0 and C_1 (Percent).

Project	STRATA	C_0	C_1	STRATA raw	vs. C_0	vs. C_1
zlib	15.8	18.9	16.6	16.7	16.6	5.3
cJSON	30.0	35.9	31.8	31.8	16.4	5.5
littlefs	20.0	24.2	21.4	21.5	17.5	6.7
FreeRTOS	44.0	53.6	47.8	47.8	17.9	7.9
Lua	35.9	43.4	38.1	38.1	17.3	5.7
Mbed TLS	80.0	96.9	87.9	87.9	17.4	9.0
TinyCC	156.5	189.8	168.2	168.3	17.5	7.0
SQLite	140.4	169.4	152.8	152.9	17.2	8.1

TABLE 9. Smallest Fixed Buffer (KiB) of Five Layouts and the Savings (Percent) of Dropping the Stored Hash ($C_0 \to C_1$), of Packing ($C_1 \to S$) and in Total ($C_0 \to S$).

trace	C_0	C_1	C_2	S_{raw}	S	hash	pack	total
zlib	18.9	16.6	16.6	16.7	15.8	11.9	5.3	16.6
cJSON	35.9	31.8	31.8	31.8	30.0	11.5	5.5	16.4
littlefs	24.2	21.4	21.4	21.5	20.0	11.6	6.7	17.5
FreeRTOS	53.6	47.8	47.8	47.8	44.0	11.0	7.9	17.9
Lua	43.4	38.1	38.1	38.1	35.9	12.2	5.7	17.3
Mbed TLS	96.9	87.9	87.9	87.9	80.0	9.2	9.0	17.4
TinyCC	189.8	168.2	168.2	168.3	156.5	11.3	7.0	17.5
SQLite	169.4	152.8	152.8	152.9	140.4	9.8	8.1	17.2

isolated: the frequency of the pinned core at the start of a session ranged from 0.4 to 4.2 GHz, its sibling hardware thread was 1.2 to 7.7 percent busy, and the load average at the start was 0.3 to 3.9; absolute times differed by up to a factor of 2.0 between sessions, so only ratios are interpreted. The unit of replication is the session. For each pair of tables and trace we give the pooled median ratio of paired times with a 95 percent t interval over the five session medians (4 degrees of freedom). "Slower" or "faster" means that the interval excludes 1. To test equivalence we use two one-sided tests with a stated practical margin, a ratio in $[0.90, 1.10]$: "eq" if the 90 percent interval lies inside the margin, "out" if it lies outside, "inc" otherwise. The margin is a choice, not derived from the data. The comparisons are exploratory, not confirmatory; 32 intervals are reported without correction for multiplicity, and the eight traces are correlated through shared compiler behavior, so they are not eight independent samples of programs.

The result differs from the first run (Table 12). Against the conventional arena table STRATA is slower on 8 of 8 traces, by 5 to 18 percent, and for six of them the equivalence margin of ± 10 percent is excluded. Against the stronger baseline C_1 it is slower on 6 of 8 traces by 4 to 13 percent, faster on 1 (zlib, by 2 percent) and not established on SQLite, and equivalent within the margin on 6. The conventional table without the hash word is itself slower than with it on 8 of 8 traces, by 3 to 9 percent. Against METIS-X, STRATA is faster on 8 of 8 traces, by 9 to 31 percent; the keyed hash costs 19 to 30 percent. The sign of the difference to the arena table therefore did not

TABLE 10. Replay Time on the Eight Compiler Traces: Median Ratio Over 31 Paired Rounds, in Percent, With 95 Percent Bootstrap Interval. Negative Means That the First Table Is Faster; "n.e." Means Not Established.

Project	STRATA vs. arena table		STRATA vs. METIS-X		Arena table vs. METIS-X		Keyed vs. default hash
cJSON	+0.2 [−3.9, +1.6]	n.e.	−22.8 [−27.5, −19.1]	faster	−21.3 [−23.8, −19.8]	faster	+20.2 [+18.5, +24.6]
littlefs	+7.2 [+2.4, +11.0]	slower	−24.5 [−27.2, −18.1]	faster	−26.5 [−28.9, −22.7]	faster	+24.6 [+18.5, +29.8]
zlib	−10.6 [−15.6, +4.1]	n.e.	−29.9 [−32.8, −27.4]	faster	−20.4 [−26.9, −15.6]	faster	+32.2 [+16.1, +41.4]
Lua	−2.7 [−13.3, +3.6]	n.e.	−30.9 [−35.3, −29.2]	faster	−28.4 [−33.8, −25.2]	faster	+22.7 [+19.1, +35.2]
FreeRTOS	+0.4 [−4.0, +6.6]	n.e.	−33.9 [−39.0, −31.3]	faster	−34.8 [−36.5, −32.9]	faster	+20.3 [+15.2, +22.4]
Mbed TLS	+0.2 [−2.8, +3.3]	n.e.	−36.7 [−39.6, −32.4]	faster	−34.8 [−38.3, −32.2]	faster	+27.1 [+23.0, +34.3]
TinyCC	+10.7 [+5.9, +12.7]	slower	−20.4 [−25.7, −18.7]	faster	−27.5 [−33.7, −22.3]	faster	+20.6 [+13.1, +22.0]
SQLite	−1.1 [−3.7, +2.1]	n.e.	−15.9 [−20.7, −6.3]	faster	−14.6 [−19.5, −10.0]	faster	+22.3 [+18.7, +33.8]

TABLE 11. 95th-Percentile Latency per Operation in Nanoseconds for STRATA (S), the Conventional Arena Table (A) and METIS-X (M).

	Lookup			Declare			Scope exit		
Project	S	A	M	S	A	M	S	A	M
cJSON	49	41	49	62	71	141	13	19	56
littlefs	51	41	49	65	72	187	19	26	90
zlib	44	41	44	56	68	136	16	23	59
Lua	53	89	54	66	130	164	21	56	84
FreeRTOS	52	44	54	68	73	180	15	23	87
Mbed TLS	66	54	69	79	78	202	27	36	109
TinyCC	51	44	51	66	75	188	20	24	76
SQLite	52	53	65	71	86	204	22	28	76

reverse; in the first run no difference was established on most traces, and that holds for that run only; the first run and these sessions differ in compiler, kernel, processor and scheduling, and we cannot separate these factors. The practical conclusion is that the compact table is not faster than a conventional single-buffer table, and that in these sessions it is slower by 5 to 18 percent against the table with a stored hash and by up to 13 percent against the one without.

Measurement details. A replay time is one interval of the C++ steady clock (`clock_gettime` with `CLOCK_MONOTONIC`, 1 ns resolution) around the replay of a whole trace, with no batching; table creation for each translation unit is inside the interval. The empty interval costs a median of 20 ns (18 ns for raw `clock_gettime`, 13 ns for `rdtscp`), negligible against replay times of hundreds of microseconds or more. Per-operation latencies (13 to 79 ns) are taken with `rdtscp` with the median empty-pair overhead subtracted, which adds a fixed cost to every operation and may change the measured cost of the instrumented operation; the 95th percentile is taken over all samples of a whole trace pooled across translation units, as the median of three replays, so a translation unit with many operations weighs more. Raw paired rounds, the per-session context (load, frequency, busy fractions) and the exact commands are published with the artifact.

Instruction counts. Wall-clock time on a shared machine depends on clock frequency and load. We therefore also counted the user-space instructions that a replay executes, with the processor's performance counters (`benchmarks/instr_count.cpp`). Such a count does not depend on frequency or on other programs: the five replays of a table on a trace differed by at most 0.03 percent (Table 13). STRATA executes 31 to 41 percent more instructions than C_0 and 21 to 29 percent more than C_1 on every trace, and the keyed variant 47 to 58 percent more than unkeyed STRATA. Counting the instructions of each step of a declaration locates most of the excess in the capacity check with its amortized index rebuilds, which must hash every name again because no hash is stored, and in writing the record, the index slot and the undo entry; with raw instead of packed names the count falls by only 8 to 9 percent. Each of these steps pays for a property of the design (the smaller buffer, exact undo, refusal without change), so we did not remove them. Against C_0 the count agrees in sign with the repeated sessions, but not in size: the extra instructions cost 5 to 18 percent of time, which we attribute, as a hypothesis we did not test with cycle or cache counters, to memory accesses that are common to all tables. Against C_1 the counts do not predict the times: zlib executes 22 percent more instructions than C_1 and is 2 percent faster. An instruction count belongs to one build: it does not depend on the clock or the load, but it does depend on the compiler and the instruction set, so these x86-64 counts say nothing direct about the Thumb-2 code of the microcontroller; it is not a time, and we do not convert it into one. Preparing the name is a large part of the work of both tables. When both tables are given names that were packed and hashed once before the replay (`benchmarks/prepared_count.cpp`; construction and destruction of the table not counted), STRATA uses 38 to 45 percent fewer instructions and C_0 about one third fewer, and the excess of STRATA over C_0 falls from 43–54 to 18–44 percent (18 percent on SQLite, 35 to 44 percent on the others); the plain figure differs from the 31 to 41 percent above because this harness does not count the creation and destruction of the table for each translation unit. This is a property of the interface (a compiler's lexer can prepare a name once), not a result about the tables as called here, and the time cost of the remainder was not measured. Three small changes (one validity test per packed word with a compile-time character table, a fast path for the capacity check, and a plain loop in place of a library call for the short record copy) lowered the counts of STRATA by 6 to 8 percent, with identical results. The timing sessions and the instruction counts of the table above were made with the final code; the emulated microcontroller run was made before these changes and could not be repeated, since the ARM toolchain was not available, so its code-size and stack figures are those of the

TABLE 12. Replay Time in Five Independent Sessions: Pooled Median Ratio of Paired Times With 95 Percent t Interval Over Sessions, Verdict (Interval Excludes 1 or "n.e.") and Equivalence Test Against ±10 Percent (eq, out, inc). Ratio Below 1 Means That the First Table Is Faster.

	STRATA / C_0			STRATA / C_1			C_1 / C_0			STRATA / METIS-X		
Project	Ratio [95% CI]	Verdict	Eq.	Ratio [95% CI]	Verdict	Eq.	Ratio [95% CI]	Verdict	Eq.	Ratio [95% CI]	Verdict	Eq.
zlib	1.049 [1.040, 1.059]	slower	eq	0.977 [0.969, 0.990]	faster	eq	1.072 [1.061, 1.082]	slower	eq	0.753 [0.749, 0.758]	faster	out
cJSON	1.140 [1.137, 1.144]	slower	out	1.046 [1.041, 1.051]	slower	eq	1.093 [1.085, 1.099]	slower	eq	0.823 [0.819, 0.827]	faster	out
littlefs	1.145 [1.118, 1.160]	slower	out	1.085 [1.055, 1.112]	slower	inc	1.052 [1.038, 1.065]	slower	eq	0.816 [0.808, 0.821]	faster	out
FreeRTOS	1.146 [1.136, 1.154]	slower	out	1.068 [1.063, 1.072]	slower	eq	1.074 [1.065, 1.080]	slower	eq	0.710 [0.706, 0.713]	faster	out
Lua	1.127 [1.111, 1.140]	slower	out	1.055 [1.044, 1.064]	slower	eq	1.069 [1.062, 1.074]	slower	eq	0.754 [0.750, 0.756]	faster	out
Mbed TLS	1.119 [1.102, 1.131]	slower	out	1.036 [1.028, 1.047]	slower	eq	1.077 [1.071, 1.082]	slower	eq	0.686 [0.682, 0.689]	faster	out
TinyCC	1.182 [1.162, 1.195]	slower	out	1.131 [1.113, 1.144]	slower	out	1.044 [1.038, 1.051]	slower	eq	0.892 [0.880, 0.900]	faster	out
SQLite	1.050 [1.028, 1.069]	slower	eq	1.020 [0.995, 1.037]	n.e.	eq	1.030 [1.023, 1.037]	slower	eq	0.911 [0.909, 0.913]	faster	eq

TABLE 13. User-Space Instructions for One Replay of Each Trace (Millions, Median of Five Replays; x86-64, `g++ -O2`) and the Ratio of STRATA to the Two Conventional Tables. The Count of a Table Varied by at Most 0.03 Percent Between Replays.

Project	STRATA	C_0	C_1	STRATA/C_0	STRATA/C_1
zlib	13.78	9.99	11.30	1.38	1.22
cJSON	4.74	3.46	3.88	1.37	1.22
littlefs	4.83	3.42	3.74	1.41	1.29
FreeRTOS	10.08	7.37	8.17	1.37	1.23
Lua	47.38	33.63	37.55	1.41	1.26
Mbed TLS	280.60	204.98	226.47	1.37	1.24
TinyCC	19.71	13.98	15.26	1.41	1.29
SQLite	53.75	41.03	44.33	1.31	1.21

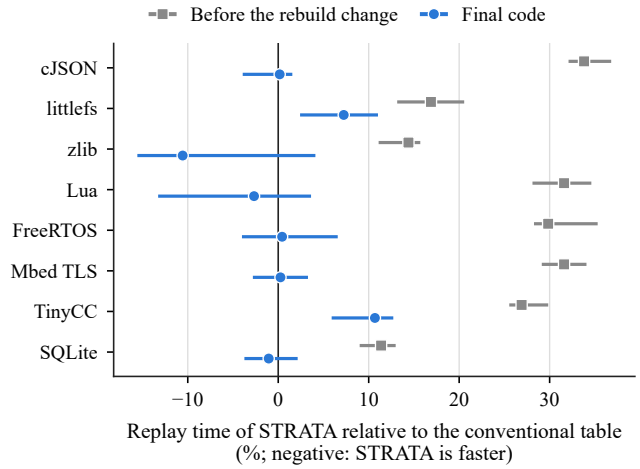


FIGURE 10. Replay time of STRATA relative to the conventional arena table before and after the rebuild change of Section IV-E: median of paired ratios with 95 percent bootstrap interval.

earlier code. Two further attempts gave no gain and were dropped: a cheaper hash step, and a one-entry cache of the last lookup, which hit on 17 to 28 percent of lookups but cost as much on the misses as it saved on the hits. Because the count is reproducible to within 0.03 percent, it is not subject to the instability that affected the timing verdicts.

F. AN EARLIER TIME PENALTY WAS AN IMPLEMENTATION ARTIFACT

An earlier version of STRATA was slower than the conventional arena table on every trace, by 11.4 to 33.8 percent (Fig. 10). Per-operation latencies showed that lookups were

TABLE 14. Ablation: Change Relative to Default STRATA in Smallest Fixed Buffer and in Replay Time (Percent; Verdict From the 95 Percent Interval).

	Raw names		Sparse index		No shrinking
Project	Buffer	Time	Buffer	Time	Time
cJSON	+5.8	−0.2 (n.e.)	+26.7	−12.1 (faster)	−2.8 (faster)
littlefs	+7.5	+4.4 (n.e.)	+20.0	−12.9 (faster)	+0.7 (n.e.)
zlib	+6.0	+3.1 (n.e.)	+25.4	−9.0 (faster)	−4.9 (faster)
Lua	+6.1	+7.1 (slower)	+22.3	−11.8 (faster)	+1.4 (n.e.)
FreeRTOS	+8.5	+3.0 (n.e.)	+18.2	−9.3 (faster)	−4.4 (n.e.)
Mbed TLS	+9.9	−1.6 (n.e.)	+20.0	−4.8 (n.e.)	+2.7 (slower)
TinyCC	+7.5	+9.9 (slower)	+20.4	−9.2 (faster)	+1.2 (n.e.)
SQLite	+8.9	−0.2 (n.e.)	+22.8	−4.5 (n.e.)	+2.9 (n.e.)

level and the whole difference was in declarations, caused by redundant rebuild work (rebuilding on every buffer growth, hashing each name twice per rebuild, and a second probe walk for a new name). The changes of Section IV-E remove it and alter no result other than time. We keep the episode because the first measurement supported a plausible but wrong conclusion.

G. ABLATION AND SYNTHETIC TRACES

Table 14 isolates three design choices. Storing names as raw bytes instead of packing them needs a 5.8 to 9.9 percent larger buffer and is slower on 2 of 8 traces and faster on 0, because a packed name has fewer words to hash and compare. Keeping the index at most half full makes replay faster on 6 of 8 traces (changes between −4.5 and −12.9 percent) and needs a 18 to 27 percent larger buffer, which more than cancels the memory advantage: index load is the lever that trades the saving for speed. On the 9 synthetic traces (Table 15) STRATA is faster than the conventional table on 6 and slower on 1: faster where names are hidden often, where scopes are entered and left at a high rate, and with short names, and slower on the trace dominated by lookups.

H. HASH FLOODING AND ITS DEFENSES

We declare and then look up the 32 768 colliding identifiers and as many random identifiers of the same length (Fig. 11, Table 16). In the default configuration the colliding names take 1954 ms to declare against 6.5 ms for ordinary names, a factor of 300, and 1684 ms to look up; time grows with exponent 1.96 in the number of names, since every new name scans the cluster formed by all earlier ones. With the keyed

TABLE 15. Synthetic Stress Traces: Replay Time of STRATA Relative to the Conventional Arena Table and to METIS-X (Percent, With Verdict), and Whether METIS-X Agrees With the Reference Model.

Trace	Declarations	Maximum depth	STRATA vs. arena table	STRATA vs. METIS-X	METIS-X result
`big_scope_exit`	200 100	1	−39.0 (faster)	−53.6 (faster)	agrees
`deep_nesting`	89 949	200	−12.1 (faster)	−32.1 (faster)	agrees
`deep_shadow_400`	400	400	−51.3 (faster)	−96.4 (faster)	149 refused
`dense_50k`	50 000	0	+0.7 (n.e.)	−26.8 (faster)	agrees
`long_ids`	89 899	10	+6.9 (n.e.)	−38.9 (faster)	agrees
`lookup_heavy`	15 097	10	+12.5 (slower)	−0.1 (n.e.)	agrees
`scope_churn`	150 050	1	−24.7 (faster)	−26.3 (faster)	agrees
`shadow_heavy`	134 903	40	−21.2 (faster)	−54.3 (faster)	agrees
`short_ids`	89 646	10	−16.3 (faster)	−36.0 (faster)	agrees

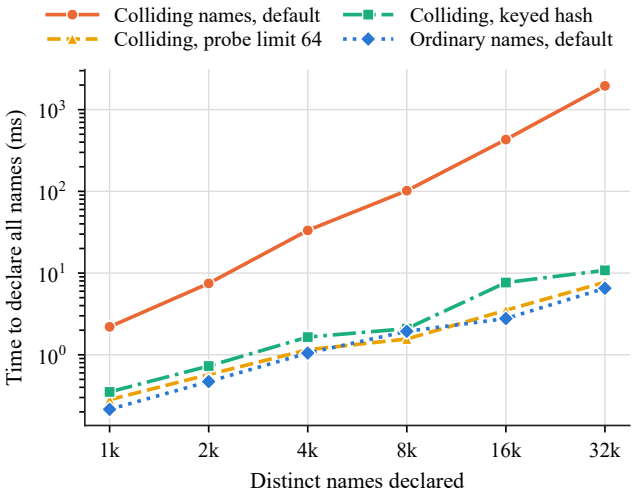


FIGURE 11. Time to declare *N* distinct names of 150 characters. Colliding names are those of Section VI-D; ordinary names are random identifiers of the same length. Both axes are logarithmic.

hash the factor is 1.2: names built against the unkeyed mixer are ordinary names under a key their author does not know. With the unkeyed hash and a probe limit of 64, time grows linearly, at the price that 32 704 of the 32 768 colliding names are refused and only 64 are admitted: the capped table finishes quickly because it does almost no useful work, and its time is not comparable with that of a table that stores all names. The limit bounds the work of a call whatever the hash does. To see what it costs honest names, we declared 1 000 colliding and 1 000 honest names in one bounded table of 4 096 slots, 20 times per setting, in random order and with the flood first (the worst case). With the unkeyed hash and $L = 64$, a mean of 2.5 honest names (at most 6) were refused in random order and 2.1 (at most 5) with the flood first, since the table stores only about 64 colliding names; with $L = 512$ the means were 5.8 (at most 17) and 16.3 (at most 26) of 1 000, at most 2.6 percent. With the keyed hash none of the 80 runs refused an honest name. The limit therefore confines the damage to a few honest names, but does not remove it for the unkeyed hash. A keyed hash defeats the collision family built against the unkeyed mixer; that is evidence about this one family, not a proof that no collision strategy works, and a fresh key is argued under the assumptions of Section VI-C. On the

TABLE 16. Declaring 32 768 Names of 150 Characters in Four Configurations: Time in Milliseconds, Their Ratio, Refused Names and Longest Probe Sequence for the Colliding Input.

Configuration	Ordinary	Colliding	Ratio	Refused	Longest
Default	6.52	1953.86	299.7	0	32 768
Keyed hash	9.33	10.80	1.2	0	106
Limit 64	3.11	7.72	2.5	32 704	64
Limit 64, keyed	5.22	5.56	1.1	0	26

compiler traces the keyed hash increases replay time by 20.2 to 32.2 percent (Table 10).

I. THE REFUSAL LAW AGAINST THE IMPLEMENTATION AND THE TRACES

Each trial draws a fresh random 64-bit key, builds a table A without a probe limit and a table B with limit L (same key, same index size), offers both the same n random distinct names, and then offers one more fresh name. Table 17 and Fig. 12 compare the exact law with process (P2): the observed frequency, over *all* trials, that the probe count of the last name in A exceeds L, which is the event that B would refuse it after a refusal-free fill. All ten z-scores lie within ± 1.96 (largest $|z| = 1.96$). The *clean* trials are those in which B refused none of the n earlier names; they are not used for the observed rate. In them B was compared with A in lock step (equal state digests and B refuses the last name iff the probe count exceeds L), with no mismatch; this validates the equivalence of the limited and unlimited tables up to the first refusal, and says nothing about behavior after a refusal. That is the content of Table 18: for each setting it lists the trials with a refusal in the fill, the refused fill operations, the frequency of a refusal of the last name given a clean fill (P3), and the frequency of a refusal of the last name in B over all trials after it kept inserting past its refusals (P4). Both frequencies lie below the exact law in every setting (P3 by 28 to 56 percent, P4 by 2 to 20 percent), as the bounds of this section require (the bound for (P3) is weak at these settings, since $1-\sum_t R_m(t,L)$ is far below 1 there, so the agreement tests the direction and not the size of the effect), and in no trial did B refuse a name that A would have admitted within L probes ("violations" $= 0$), which is the inclusion used in the coupling argument. The settings have refusal probabilities between 4×10^{-4} and 6×10^{-2}, which 400 000 trials resolve (100 000 trials at the largest index resolve about 3×10^{-3}); the design values near 10^{-6} and 10^{-11} are not

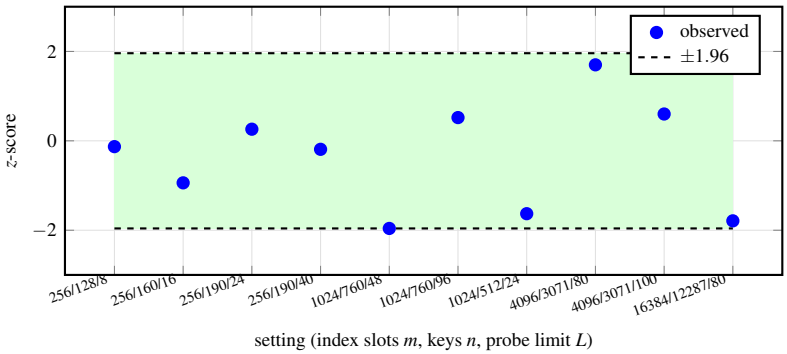


FIGURE 12. Agreement of the exact refusal law with the real bounded-mode table: standard scores in ten settings of 400 000 trials each (100 000 at 16 384 slots), all within plus or minus 1.96.

tested by trials, they rest on the formula, its enumeration and independent-derivation checks, and the stored simulation of 240 000 000 insertions (Fig. 6). The law therefore describes the first capped query of the implementation, not only of the model; the refusal-filtered process is covered by the bounds, not by an exact law. These trials use random names under a keyed hash, so they compare the implementation with the model but, taken alone, say nothing about the default mixer on real identifiers. To test that, we drew random subsets of the distinct identifiers of four compiler traces (Lua, Mbed TLS, SQLite and TinyCC, each with at least four times as many names as are stored) into an unkeyed table of 1 024 slots holding 760 names, 50 000 times each, and counted how often the next fresh name needed more than L probes. The observed frequencies were 0.0104 to 0.0145 against the law's 0.0129 at $L = 48$ (within 19 percent of it) and 0.00042 to 0.00084 against 0.00058 at $L = 96$ (21 to 42 events). The trials of one vocabulary share its names and so are not independent, and each vocabulary is one realization of the hash. To see how far one realization can deviate by chance, we repeated the draws on the same names under 100 random keyed hashes: the frequency then has a standard deviation of 12 to 19 percent of the law at $L = 48$ and 37 to 50 percent at $L = 96$, and all eight unkeyed frequencies lie within 1.1 standard deviations of the keyed mean. The check therefore shows no departure from the model on these names; it has little power to detect a small one, and it does not show that the mixer behaves randomly on every vocabulary. Settings in which n exceeds a quarter of the vocabulary (the 4 096-slot settings and the smaller vocabularies) are in the data and are excluded, because the subsets then barely differ.

The compiler traces alone would have suggested a much smaller limit. Replaying the 15 compiler traces (719 833 declarations) in bounded mode, with the index sized for each trace's peak, a limit of 64 refuses no declaration with either hash and the longest probe sequence is 47 (Table 19). Yet at the load limit of $3/4$, 0.77 percent of fresh names need more than 64 probes: the traces' tables are small and spend most of their time well below the load limit. The exact law gives the setting directly: 258 for a refusal probability of 10^{-6} per declaration at 2^{16} slots, or 52 with an index kept at most half full. The limit of 512 recommended in the library documentation is safe but conservative: at 65 536 slots it corresponds to a refusal probability of 2.4×10^{-11} per declaration, and at 4 096 slots, closer to the buffers of this study, to 10^{-12}; for a refusal probability of 10^{-6} the exact limits are 242 at 4 096 slots and 250 at 8 192, so 258 is an upper end that is reached only for large indexes.

TABLE 17. Process (P2): Exact Law Against the Real Probe-Limited Table. Observed Frequency Over All Trials That the Last Name Needs More Than *L* Probes, the Exact Value, the *z*-Score, Clean Trials (No Refusal in the Fill) and Lock-Step Mismatches in Clean Trials (*m* Index Slots, *n* Keys Stored, *L* Probe Limit).

m	n	L	trials	observed	exact	z	clean trials	mismatches
256	128	8	400 000	0.03311	0.03315	-0.13	254 871	0
256	160	16	400 000	0.02643	0.02667	-0.94	307 410	0
256	190	24	400 000	0.05740	0.05730	+0.26	247 303	0
256	190	40	400 000	0.01372	0.01375	-0.19	369 939	0
1024	760	48	400 000	0.01251	0.01286	-1.96	307 286	0
1024	760	96	400 000	0.00060	0.00058	+0.52	397 066	0
1024	512	24	400 000	0.00040	0.00046	-1.63	395 607	0
4096	3071	80	400 000	0.00317	0.00302	+1.70	335 646	0
4096	3071	100	400 000	0.00110	0.00106	+0.60	378 922	0
16384	12287	80	100 000	0.00296	0.00328	-1.79	46 819	0

TABLE 18. Processes (P3) and (P4) in the Same Trials as the Previous Table (400 000 per Setting, 100 000 at 16 384 Slots): Clean Trials, Trials With a Refusal in the Fill, Refused Fill Operations, Frequency of a Refusal of the Last Name Given a Clean Fill (P3) and in the Table That Kept Inserting After Refusals (P4), and Trials in Which the Limited Table Refused a Name the Unlimited One Admitted Within *L* Probes.

m	n	L	clean	with refusal	refused in fill	(iii) $\hat{P}$	(iv) $\hat{P}$	violations
256	128	8	254 871	145 129	210 813	0.02384	0.02931	0
256	160	16	307 410	92 590	137 811	0.01664	0.02325	0
256	190	24	247 303	152 697	282 374	0.03151	0.04899	0
256	190	40	369 939	30 061	45 241	0.00810	0.01246	0
1024	760	48	307 286	92 714	169 972	0.00624	0.01109	0
1024	760	96	397 066	2 934	4 327	0.00040	0.00057	0
1024	512	24	395 607	4 393	5 212	0.00030	0.00038	0
4096	3071	80	335 646	64 354	116 053	0.00164	0.00285	0
4096	3071	100	378 922	21 078	34 694	0.00058	0.00103	0
16384	12287	80	46 819	53 181	127 464	0.00145	0.00263	0

J. WHEN DOES THE SMALLER BUFFER MATTER?

A saving of one sixth decides whether a workload fits only when the budget lies between the needs of the two tables. Of the 169 translation units of the eight projects, the units

TABLE 19. Bounded Mode on 15 Compiler Traces (719 833 Declarations): Declarations Refused by the Probe Limit and Number of Traces Affected, for the Default and the Keyed Hash.

	Default hash		Keyed hash	
Limit	Refused	Traces	Refused	Traces
4	15 335	15	15 589	15
8	3 543	15	3 507	15
16	561	12	571	11
32	51	5	51	4
64	0	0	0	0
128	0	0	0	0
256	0	0	0	0
512	0	0	0	0

TABLE 20. Translation Units (of 169) That Fit a Fixed Budget: in Both Single-Buffer Tables, Only in STRATA, or in Neither.

Budget (KiB)	Both	Only STRATA	Neither
8	1	1	167
16	28	23	118
32	100	7	62
64	146	1	22
128	167	0	2

TABLE 21. Sweep Over the Shadowing Factor σ and the Name Length: Saving of STRATA in Smallest Fixed Buffer (Percent) and Replay Time Relative to the Conventional Arena Table (Percent).

	6 characters		12 characters		24 characters	
σ	Save	Time	Save	Time	Save	Time
1.0	0	−30.7 (faster)	0	+0.6 (n.e.)	9	+6.1 (n.e.)
1.5	24	−36.7 (faster)	25	+5.5 (n.e.)	33	+5.8 (slower)
2.0	29	−35.2 (faster)	31	+8.1 (slower)	41	+12.2 (slower)
4.0	43	−47.5 (faster)	47	−4.4 (n.e.)	57	+16.0 (slower)
8.0	50	−39.9 (faster)	55	+2.8 (n.e.)	65	+20.9 (slower)

that only STRATA can hold are few at every budget, at most 23 at 16 KiB (Table 20). On the emulated microcontroller (Section IX-L), three whole workloads fit a buffer in which the conventional table refuses operations. A sweep over the shadowing factor (live declarations per distinct name) and the name length confirms (1) and the ledger: the saving grows with shadowing, since a hidden declaration costs three words in STRATA and a full entry with the name in the conventional table, and reaches 65 percent at the highest factor tested (Table 21). Replay time relative to the conventional table follows the name length rather than the shadowing factor.

K. INSIDE A COMPILER

To see what the differences amount to in a whole compiler, we replaced the per-scope hash maps that chibicc [41] uses for variable scopes by STRATA and, separately, by the conventional arena table, and compiled seven of the projects with all three builds. The unmodified compiler accepts 158 of their 170 translation units; on those, the assembly produced with STRATA is byte-for-byte identical to the original on 158 units and with the arena table on 158. Over 21 interleaved rounds (Table 22), whole-project compile time with STRATA changes by between −3.7 and +2.2 percent, and with the arena table by between −3.1 and +4.3 percent; peak resident memory per unit changes by a median of 0.18 percent. In a hosted compiler neither single-buffer table matters: symbol-table work is a small part of the compile. With STRATA the interval excludes zero on one project, Mbed TLS, where it is slower by 1.4 percent (interval 0.2 to 2.4); the 14 intervals are not corrected for multiple comparisons, and the other medians are quoted in the range above. The experiment establishes that the tables are correct in a real resolution path; it does not show a benefit for compilers that are not memory-constrained.

L. AN EMULATED MICROCONTROLLER

We built a bare-metal image for a Cortex-M3 and ran it under QEMU with 64 KiB of memory, with allocation functions stubbed to fail. Five of the eight traces fit that memory; all 5 replay with a checksum over every result identical to the host's, using 492 bytes of stack, and the table's code occupies at least 2 715 bytes. The same 5 traces pass in bounded mode with the keyed hash and a probe limit of 512, with no refusal. At equal buffers, STRATA fits FreeRTOS in 48 KiB, Lua in 40 KiB and cJSON in 32 KiB, where the conventional table refuses 420, 2 214 and 822 operations. These budgets lie in the window between the needs of the two tables, which exists because the tables differ in size; they illustrate the window and do not estimate how often a real budget falls in it. The other three traces need more than 64 KiB and were not run. The 64 KiB limit is enforced by the linker script only, not by an emulator option; the trace input is constant data in flash and is not counted in it. The stack figure is the high-water mark of one run in fixed mode (492 bytes; 484 in bounded mode) measured by a painted stack region, and the code figure is the sum of the sizes of the `strata::` symbols in the unlinked object, not a linked image. The emulator provides no timing, and no physical device was used, so these results show that the code fits, runs and returns identical answers within the stated memory, not how fast it runs on a chip. The results that the embedded experiment supports, that the table fits, runs without a heap and gives the host's answers within the stated memory, and the mathematical results of Sections V and VI, do not depend on a physical device; a claim about speed on a device would, and none is made. The instruction counts of Section IX-E are of the x86-64 host build and are not a substitute for such a timing.

X. AN INTERACTIVE TOOL

To make the design inspectable, the artifact includes a web interface (React [44] and Vite [45]) whose local server runs a C++ engine linked with the unmodified header. Every request replays the whole list of operations on a fresh table, so the C++ table is always the source of truth; after each request the engine checks the table's invariants, that every decoded record equals what `resolve` returns, and that every answer equals the reference model. The *Analyse your code* page (Fig. 13) accepts a C program, translates it into declare, lookup, enter and exit calls, and reports every name, its scope and uses, shadowing, and the smallest buffer from 256 B to 64 KiB that refuses nothing. A *Live lab* shows the buffer decoded word by word after each call and the digest before and after a refused

TABLE 22. End to End in chibicc: Whole-Project Compile Time With Variable Scopes on STRATA or on the Conventional Arena Table, Relative to the Original Compiler (Percent, 95 Percent Interval; Negative Is Faster).

Project	Units compiled	Original compiler (s)	STRATA		Arena table	
cJSON	2	0.11	−0.7 [−5.9, +8.8]	n.e.	+4.3 [−2.6, +10.7]	n.e.
littlefs	1	0.02	−3.7 [−5.8, +2.6]	n.e.	−3.1 [−6.4, −0.7]	faster
zlib	12	0.30	+2.2 [−3.0, +5.7]	n.e.	+0.5 [−3.9, +4.9]	n.e.
Lua	34	1.09	−0.7 [−3.5, +0.5]	n.e.	−1.8 [−2.7, −0.3]	faster
FreeRTOS	7	0.65	+1.8 [−4.1, +5.5]	n.e.	−0.5 [−7.0, +7.8]	n.e.
Mbed TLS	101	5.58	+1.4 [+0.2, +2.4]	slower	+1.0 [−0.7, +1.5]	n.e.
SQLite	1	1.00	+0.0 [−3.3, +2.8]	n.e.	−2.1 [−3.6, −0.2]	faster

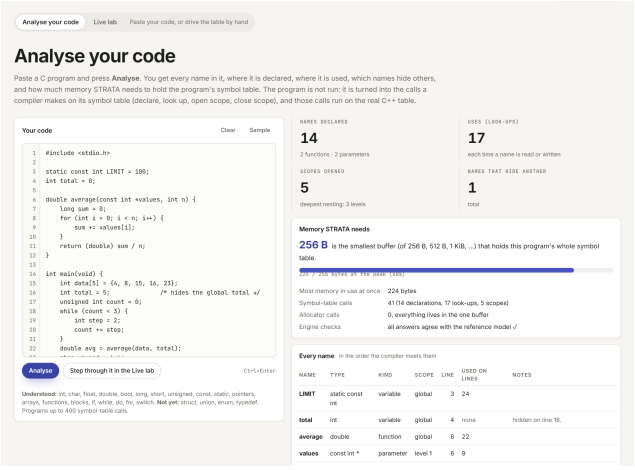


FIGURE 13. The "Analyse your code" page. A pasted C program is translated into the calls a compiler makes on its symbol table, which run on the real STRATA table in a local engine; the page lists every name with its type, scope, line and uses, the names it hides, unused and undeclared names, and the smallest buffer that holds the program.

call, and the results and theory pages draw every figure from the stored measurements and recompute the refusal law in the browser. The interface changes neither the library nor any result; its 37 automated checks compare it with the engine and with the Python programs of the analysis.

XI. DISCUSSION

What the compact design buys. Against a conventional chained table that obeys the same storage discipline and stores a hash word, STRATA offers a buffer about one sixth smaller; against the same table without the hash word, which we implemented and measured, the buffer is 5.3 to 9.0 percent smaller, all of it from name packing. In the first run no difference in speed was established on most traces; in five repeated sessions on another configuration STRATA was slower than the conventional table with the hash word on every trace (5 to 18 percent) and than the one without it on most. A conventional table can obtain most of the saving by dropping its stored hash, at a cost in time of its own. For a compiler on a workstation neither table matters. For a front end with tens of kilobytes, one sixth of the symbol table is memory that can go elsewhere, and on the emulated microcontroller it decided whether three workloads fit.

What the storage discipline buys. One buffer and write-after-reserve give a refusal that is simple to state and simple to test exhaustively at small scale. We consider this the more transferable result; it applies equally to the conventional table.

Why last-in-first-out removal matters beyond speed. The removal rule was adopted to avoid tombstones. Its more useful consequence is Theorem 1: because the index is always that of an insertion-only table, the classical exact analysis of linear probing applies to it, in the ideal-hash model and up to the first refusal, at every moment, and Theorem 2 shows that for linear probing no other deletion order allows this for every table.

Why measurement alone was not enough. Three times in this work a first measurement supported a conclusion that a closer measurement or the analysis overturned: the size of the time penalty of Section IX-F (found by per-operation profiling), the probe limit of 64 that the traces suggested (overturned by the refusal law), and the reading of the buffer saving as a benefit of interning (overturned by the ledger). These are arguments for pairing replay experiments with a baseline that shares the design's constraints and with an exact analysis of the quantity being tuned.

Where the design is the wrong choice. If memory is not constrained and the input is trusted, this study gives no reason to prefer a single-buffer table. If the input is untrusted, the keyed hash costs 20.2 to 32.2 percent of replay time, and its guarantee requires a fresh key per table.

XII. LIMITATIONS AND THREATS TO VALIDITY

Mostly replay, not compilation. The traces record what a compiler's own tables were asked to do; replaying them measures a table, not a compiler built on it. The one experiment inside a compiler replaces variable scopes only, on a hosted machine, and shows no end-to-end benefit.

Two machine configurations, no hardware. Timing comes from two configurations, a laptop under a hypervisor and a shared native desktop, and the microcontroller is emulated. The verdicts on individual traces are not stable across runs: in an earlier run of the same code (kept under the git tag `pre-cleanup`) STRATA was faster than the conventional table on two traces and slower on three, and in the repeated sessions it was slower than the table with a stored hash on all eight. The sessions are not on an isolated machine (frequency scaling, a lightly loaded sibling thread). The traces are correlated, the intervals are exploratory and uncorrected for multiplicity, and the equivalence margin of ± 10 percent

is a choice. On the eight compiler traces the timing supports only the claim that the compact table is not faster than a conventional single-buffer table; the synthetic traces and the sweep, which are older measurements and compare with C_0 only, show it faster where names are short and often hidden.

Workloads. The traces are C only, from two small compilers, and depend on the host's system headers. Names outside the identifier alphabet, as in mangled C++ names, take the unpacked path, which the traces exercise little.

Baselines. The single-buffer baselines that decide the headline saving and the timing comparison, C_0 and C_1, were written by the authors of the table under study and are not independent implementations; we matched their hash, growth step and refusal behavior to STRATA's, but not the load factor: the arena tables double their buckets when the live declarations exceed the buckets (load 1 per declaration), whereas STRATA doubles its index at 3/4 per distinct name. A conventional table with packed names, which could capture the packing saving, was not built. The heap baselines wrap independently maintained containers (the C++ standard library and the parallel-hashmap Swiss table [42]), but they do not obey the single-buffer discipline, so they bear on the allocation question only. LLVM's own `ScopedHashTable` was not measured.

The analysis. The proofs are pen-and-paper, checked by exhaustive enumeration and by comparison with the implementation, not by a proof assistant. The refusal law assumes an ideal hash; against an adversary it assumes that HalfSipHash with one and three rounds and a 64-bit key behaves as one and that keys are fresh. The key-lifetime experiment uses an idealized oracle; no timing channel was measured. A limit law for refusals on wide and deep scope trees is open.

Design limits. Names of at most 255 bytes, 16 383 nested scopes, a buffer of at most 64 MiB, one name space per table, single-threaded use.

Literature and motivation. The search for related work was limited, and several sources were consulted at the level of abstracts or documentation. The three requirements of Section I are stated from the design problem; we do not cite a deployed compiler that needs all of them.

XIII. CONCLUSION

A scoped symbol table can be confined to one fixed buffer, refuse cleanly when the buffer is full, and, in bounded mode, bound the index slots examined by each declaration and lookup on hostile input (scope exit and the default-mode rebuild are not bounded this way), at a cost that we have measured and with a refusal probability that we can compute in the ideal-hash model. On events logged by a C compiler for eight projects, the compact design studied here needs a buffer 5 to 9 percent smaller than a conventional single-buffer table without a stored hash word, which we implemented and measured, and about one sixth smaller than one that stores it; an exact ledger attributes the saving to the stored hash word and to name packing, and packing could be added to a conventional table. In time it is not faster: slower than the conventional table with a stored hash on all eight traces, and than the one without on most, in repeated sessions on one configuration, with no difference established on most traces in an earlier run. Both are well ahead of a multi-allocation predecessor, and that larger gain belongs to the storage discipline. Because declarations are removed in the reverse order of their creation, the index is always that of an insertion-only table; for linear probing this is the only deletion order with this property for every table, and it makes the refusal probability of a probe limit exactly computable in the ideal-hash model: at load three quarters and 65 536 index slots, a refusal probability of one in a million per request needs 258 probes, not the 455 a tail bound suggests, and the law agrees with the implementation for the first capped query, with proved bounds for the process after refusals. Identifiers that collide for every seed of the default hash slow declarations 300-fold at 32 768 names (and growing quadratically), and a keyed hash removes the effect for that collision family; under the stated assumptions a fresh key per table keeps the refusal law valid against an oblivious adversary.

The most useful next steps are a compiler that runs on the table on a physical microcontroller, workloads from languages other than C, a limit law for refusals on deep scope trees, and machine-checked proofs.

APPENDIX A PROOF OF THE EXACT REFUSAL LAW AND OF THE BOUNDS FOR THE REFUSAL-FILTERED PROCESS

Let $k = m-n \geq 1$. For $k = 1$ one cell is empty and uniformly located, and the L cells from a uniform start are occupied iff the start lies among the $(m-L)^+$ cells that precede the empty cell by at least L. Assume $k \geq 2$ and let X_j be the number of keys with home j; (X_j) is multinomial with probability proportional to $\prod_j 1/x_j!$.

Step 1: blocks. Since $n < m$ there are exactly k empty cells, and the occupied set does not depend on the insertion order [46]. Cut the ring at the empty cells: a *block* is an empty cell together with the occupied cells after it up to the next empty cell, so a block of g cells has $g-1$ occupied ones. No key crosses an empty cell, so the keys homed in a block occupy exactly its cells. With home counts y_u of the $g-1$ occupied cells, the cells are occupied and the next one is empty iff $\sum_{t\leq u} y_t \geq u$ for $u < g-1$ and $\sum_{t\leq g-1} y_t = g-1$. Conversely, a cyclic concatenation of such blocks has exactly these empty cells, so home-count vectors correspond bijectively to placements of k blocks around the ring.

Step 2: weight of a block. Put $y_g = 0$ and $z_t = y_t - 1 \geq -1$. The condition says that the walk $z_1 + \cdots + z_u$ stays nonnegative for $u < g$ and first reaches -1 at $u = g$. By the cycle lemma [36], exactly one of the g cyclic rotations of a sequence in $\mathbb{N}^g$ with sum $g-1$ has this property. The weight $\prod_t 1/y_t!$ is rotation invariant and $\sum_{\sum y = g-1} \prod_t 1/y_t! = g^{g-1}/(g-1)!$ by the multinomial theorem, so the total weight of a block of g cells is $B(g) = g^{g-1}/g!$.

Step 3: the block of a cell. The weight of a ring configuration is the product of its block weights. Cell 0 lies in a block of g cells at one of g positions; the remaining $k-1$ blocks contain

$m-g$ cells with total weight $Q_{k-1}(m-g)$, where $Q_j(s) = \sum_{g_1+\cdots+g_j=s} \prod B(g_i) = \frac{j}{s}\, s^{s-j}/(s-j)!$ by the Borel–Tanner identity [37]. By exchangeability, $\sum_g g\, B(g) Q_{k-1}(m-g) = \frac{m}{k} Q_k(m)$, hence cell 0 lies in a block of g cells with probability $\frac{k}{m}\, g\, \pi_g$, $\pi_g = B(g)Q_{k-1}(m-g)/Q_k(m)$, at a uniform position.

Step 4: distance to the next empty cell. For position $o \in \{2,\ldots,g\}$ of a block of g cells the next empty cell is $g+1-o$ steps away, so the L cells from a start are all occupied iff $o \le g+1-L$, which holds for $(g-L)^+$ positions; position 1 is empty. Therefore the probability that the first L cells from an independent uniform home are occupied is $\sum_g \frac{k}{m} g\, \pi_g\, (g-L)^+/g = \frac{k}{m} \sum_{g>L} (g-L)\pi_g$, with $g \le n+1$.

Bounds for (P3) and (P4). (P3): $\mathbb{P}(\text{refused} \mid \text{clean}) = \mathbb{P}(\text{refused} \wedge \text{clean})/\mathbb{P}(\text{clean}) \le R_m(n,L)/P_{\text{fill}}$, and $P_{\text{fill}} \ge 1 - \sum_{t<n} R_m(t,L)$, because the event that some push of the fill is refused at limit L is the event that some push of the unlimited fill has probe count above L, a union of events of probability $R_m(t,L)$. (P4), insertions only: let A be the unlimited table and B the limited one, both receiving the same fresh names; we show $\mathrm{occ}(B) \subseteq \mathrm{occ}(A)$ by induction. If B refuses a name, B is unchanged and A only adds a cell. If B accepts a name at cell c_B, the cells of its probe path before c_B are occupied in B and therefore in A, so A places the name at a cell c_A at or after c_B on the path, and $c_B \in \mathrm{occ}(A) \cup \{c_A\}$. If B refuses a name, its first L cells are occupied in B and therefore in A; so B refuses only if the probe count in A exceeds L. The home of the offered name is independent of the table, hence the probability of refusal at the tth offer is at most $R_m(t-1,L)$.

DATA AND CODE AVAILABILITY

The implementation, the tests, the compiler patches, the traces, the raw measurements of every run cited here, the verification programs of the analysis, the interactive tool, and the scripts that regenerate every number, table and figure of this article are public [38]. The artifact contains a manifest that maps every table, figure and headline number of this article to the command that produces it, with expected run times (`docs/EVIDENCE.md`, part 7), SHA-256 checksums of every trace file, the raw paired rounds and per-session context of Section IX-E (`results/review/`), and the commit and the git tag `pre-cleanup` of the state before the repository was cleaned; the exact release to cite is fixed when the article is accepted. The enumeration of Section VII needs about 3 minutes, the refusal-law check against the real table about 15 minutes, the timing sessions about one hour including gaps, and the full benchmark pipeline several hours on one core. The formula of Theorem 3 is implemented in exact rational arithmetic (Python, compared with brute force for $m \le 9$) and in floating point (Python and TypeScript); the TypeScript version is checked against the Python values, which is a transcription check and not an independent derivation.

ACKNOWLEDGMENT

Funding. This work received no external funding. **Competing interests.** The authors declare no competing interests.

Author contributions. A. Jain and S. Singh contributed equally to the work. Prof. Bhuvaneswari M. guided the project throughout and is the corresponding author.

In accordance with the journal's policy on the disclosure of generated content: artificial-intelligence assistants (Claude, Anthropic) were used, at the direction of the authors, for planning the work, for assistance in writing the code and the verification programs, for help with the literature review, and for drafting and editing parts of the text. The tool is not an author. Every number is produced by programs that can be re-run; the proofs are pen-and-paper and were checked by exhaustive enumeration and by statistical comparison with the implementation, not by a proof assistant. All authors have read, checked and verified the content and take responsibility for all of it. A fuller account is in the repository (`docs/AI_USE_DISCLOSURE.md`).

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] S. L. Graham, W. N. Joy, and O. Roubine, "Hashed symbol tables for languages with explicit scope control," in *Proc. SIGPLAN Symposium on Compiler Construction*, 1979, pp. 50–57.

[3] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge, U.K.: Cambridge University Press, 1998.

[4] S. A. Crosby and D. S. Wallach, "Denial of service via algorithmic complexity attacks," in *Proc. 12th USENIX Security Symposium*, 2003, [Online]. Available: https://www.usenix.org/conference/12th-usenix-security-symposium/denial-service-algorithmic-complexity-attacks.

[5] J.-P. Aumasson and D. J. Bernstein, "SipHash: A fast short-input PRF," in *Progress in Cryptology – INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 489–508.

[6] LLVM Project, "ScopedHashTable.h, LLVM ADT library," [Online]. Available: https://llvm.org/doxygen/ScopedHashTable_8h_source.html. Accessed: Oct. 13, 2026.

[7] F. Bellard and TinyCC contributors, "Tiny C Compiler, commit 43c7708," [Online]. Available: https://repo.or.cz/tinycc.git. Accessed: Oct. 11, 2026.

[8] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[9] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *Proc. IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, 2021, pp. 1171–1182.

[10] M. Farach-Colton, A. Krapivin, and W. Kuszmaul, "Optimal bounds for open addressing without reordering," in *Proc. 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024, pp. 594–605.

[11] C. W. Fraser and D. R. Hanson, *A Retargetable C Compiler: Design and Implementation*. Redwood City, CA: Addison-Wesley, 1995, iSBN 0-8053-1670-1.

[12] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, 1990.

[13] MicroPython Project, "MicroPython string interning," [Online]. Available: https://docs.micropython.org/en/latest/develop/qstr.html. Accessed: Oct. 13, 2026.

[14] R. M. Jiménez and C. Martínez, "On deletions in open addressing hashing," in *Proc. 15th Workshop on Analytic Algorithmics and Combinatorics (ANALCO)*, 2018, pp. 23–31.

[15] P. Sanders, "Hashing with linear probing and referential integrity," arXiv:1808.04602, 2018.

[16] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (SFCS)*, 1985, pp. 281–288.

[17] A. Jain and S. Singh, "METIS: Symbol-table research repository, commit 7b3dacf," 2026, [Online]. Available: https://github.com/adhyan-jain/Metis. Accessed: Oct. 11, 2026.

[18] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
[19] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Distributed Computing*, ser. Lecture Notes in Computer Science. Springer, 2008, pp. 350–364.
[20] M. Naor and V. Teague, "Anti-persistence: History independent data structures," in *Proc. 33rd Annual ACM Symposium on Theory of Computing (STOC)*, 2001, pp. 492–501.
[21] G. E. Blelloch and D. Golovin, "Strongly history-independent hashing with applications," in *Proc. 48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2007, pp. 272–282.
[22] P. Flajolet, P. Poblete, and A. Viola, "On the analysis of linear probing hashing," *Algorithmica*, vol. 22, no. 4, pp. 490–515, 1998.
[23] S. Janson, "Asymptotic distribution for the cost of linear probing hashing," *Random Structures & Algorithms*, vol. 19, no. 3–4, pp. 438–471, 2001.
[24] S. Janson and A. Viola, "A unified approach to linear probing hashing with buckets," *Algorithmica*, vol. 75, no. 4, pp. 724–781, 2016, section 11 read in the arXiv version 1410.5967.
[25] S. Janson, "Individual displacements for linear probing hashing with different insertion policies," *ACM Transactions on Algorithms*, vol. 1, no. 2, pp. 177–213, 2005.
[26] A. Viola, "Exact distribution of individual displacements in linear probing hashing," *ACM Transactions on Algorithms*, vol. 1, no. 2, pp. 214–242, 2005.
[27] M. A. Bender, A. Conway, M. Farach-Colton, W. Kuszmaul, and G. Tagliavini, "Tiny pointers," *ACM Transactions on Algorithms*, vol. 21, no. 4, pp. 1–43, 2025, theorem numbers refer to the arXiv version 2111.12800, which was read.
[28] A. Alharbi, C. Banderier, and C. Bouillaguet, "Bounded linear probing hashing," in *37th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA 2026)*, ser. LIPIcs, vol. 381, 2026, pp. 28:1–28:17, abstract, introduction, Section 3 and conclusion read.
[29] A. Pagh, R. Pagh, and M. Ružić, "Linear probing with constant independence," in *Proc. 39th Annual ACM Symposium on Theory of Computing (STOC)*, 2007, pp. 318–327.
[30] M. Thorup, "Linear probing with 5-independent hashing," arXiv:1509.04549, 2015, [Online]. Available: https://arxiv.org/abs/1509.04549.
[31] J.-P. Aumasson, D. J. Bernstein, and M. Boßlet, "Hash-flooding DoS reloaded: Attacks and defenses," Presentation slides, 2012, [Online]. Available: https://www.aumasson.jp/siphash/siphashdos_appsec12_slides.pdf.
[32] J. A. Donenfeld, "SipHash – a short input PRF," Linux kernel documentation, [Online]. Available: https://docs.kernel.org/security/siphash.html. Accessed: Oct. 13, 2026.
[33] R. J. Lipton and J. F. Naughton, "Clocked adversaries for hashing," *Algorithmica*, vol. 9, no. 3, pp. 239–252, 1993.
[34] M. Naor and E. Yogev, "Bloom filters in adversarial environments," in *Advances in Cryptology – CRYPTO 2015*, ser. Lecture Notes in Computer Science. Springer, 2015, pp. 565–584.
[35] Rust Project Developers, "rustc-hash: A speedy, non-cryptographic hashing algorithm used by rustc," [Online]. Available: https://github.com/rust-lang/rustc-hash. Accessed: Oct. 13, 2026.
[36] M. Dwass, "The total progeny in a branching process and a related random walk," *Journal of Applied Probability*, vol. 6, no. 3, pp. 682–686, 1969.
[37] J. C. Tanner, "A derivation of the Borel distribution," *Biometrika*, vol. 48, no. 1–2, pp. 222–224, 1961.
[38] A. Jain and S. Singh, "STRATA: Source code, traces, and measurements," 2026, [Online]. Available: https://github.com/adhyan-jain/cdesproject.
[39] A. Appleby, "MurmurHash3, in SMHasher," [Online]. Available: https://github.com/aappleby/smhasher. Accessed: Oct. 13, 2026.
[40] J.-P. Aumasson, "SipHash and HalfSipHash reference implementation," [Online]. Available: https://github.com/veorq/SipHash. Accessed: Oct. 13, 2026.
[41] R. Ueyama, "chibicc: A small C compiler, commit 90d1f7f," [Online]. Available: https://github.com/rui314/chibicc. Accessed: Oct. 11, 2026.
[42] G. Popovitch, "The parallel hashmap, version 1.4.1," [Online]. Available: https://github.com/greg7mdp/parallel-hashmap. Accessed: Oct. 13, 2026.
[43] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, 2005, pp. 41–46.
[44] Meta Open Source, "React," [Online]. Available: https://react.dev. Accessed: Oct. 20, 2026.
[45] Vite contributors, "Vite: Next generation frontend tooling," [Online]. Available: https://vite.dev/guide/. Accessed: Oct. 20, 2026.
[46] A. G. Konheim and B. Weiss, "An occupancy discipline and applications," *SIAM Journal on Applied Mathematics*, vol. 14, no. 6, pp. 1266–1274, 1966.

ADHYAN JAIN is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. Research interests include compilers, hashing and the analysis of algorithms.

SHUBHI SINGH is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. Research interests include compilers, data structures for constrained systems, and systems software.

BHUVANESWARI M. is with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.